🚀 Data-Centric AI: Job Market Analysis & Semantic Classification

Author: Paul Tuccinardi

Tech Stack: Python, Sentence-Transformers (BERT), XGBoost, SHAP, HuggingFace (Zero-Shot)

🎯 Project Overview

This project transforms a noisy, 2016-era job dataset into a modern classification pipeline. By moving from keyword-based statistics (TF-IDF) to **Neural Semantic Embeddings**, I developed a model capable of understanding the *intent* of job descriptions.

💡 Key Innovation: Data-Centric Refinement

Faced with a 61% accuracy ceiling, I utilized **Zero-Shot LLM Classification (BART-Large)** to audit the ground-truth labels. I discovered that the majority of "errors" were actually mislabeled data in the original dataset—proving the model was more accurate than the historical labels suggested.

Data Source:

[Monster.com Job Postings Dataset]

Data dictionary

Column name	Type	Description
`job_id`	string	Unique identifier for the posting (if available)
`company`	string	Employer name
`title`	string	Job title from the posting
`location`	string	Full location string (city, state)
`description`	string	Full job description text
`salary`	string/var	Raw salary string as scraped (may be range, hourly, or missing)
`date_added`	datetime	Date the job was posted (or scraped)
`job_type`	string	Tasks and skills involved in job (if available)
`category`	string	Job category or industry tag (if available)

Notes:

- Some rows do not include salary information. We will create a derived numeric `salary_annual` and flags describing the salary source/type.
- We will standardize `location` to `city, state` and create `state` flags.

```
import nltk

# List of NLTK resources required for the skill extraction code
required_nltk_resources = ['punkt', 'stopwords', 'wordnet', 'punkt_tab']

print("Checking and downloading required NLTK resources...")

for resource in required_nltk_resources:
    try:
        # Check if the resource is already available
        nltk.data.find(f'tokenizers/{resource}' if resource == 'punkt' else f'corpora/{resource}')
        print(f"✅ Resource '{resource}' is already available.")
```

```
        except LookupError:
            # If not found (LookupError is the correct exception to catch), download it
            print(f"⬇️ Downloading resource '{resource}'...")
            nltk.download(resource, quiet=True) # Use quiet=True to suppress standard output
            print(f"✨ Downloaded '{resource}'.")
        except Exception as e:
            # Catch any other unexpected errors
            print(f"⚠️ An unexpected error occurred while checking/downloading '{resource}': {e}")

    print("NLTK setup complete. You can now run the rest of your skill extraction code.")
```

```
Checking and downloading required NLTK resources...
✅ Resource 'punkt' is already available.
✅ Resource 'stopwords' is already available.
⬇️ Downloading resource 'wordnet'...
✨ Downloaded 'wordnet'.
⬇️ Downloading resource 'punkt_tab'...
✨ Downloaded 'punkt_tab'.
NLTK setup complete. You can now run the rest of your skill extraction code.
```

Imports

```
# Core EDA + preprocessing imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from wordcloud import WordCloud
```

Initial data checks and EDA goals

Goals for this section:

1. Understand data shapes and missingness.
2. Detect labelling issues for salary (ranges, hourly, non-US currency).
3. Inspect distributions for title frequency, location, and salary (where available).

```
# Load data
df = pd.read_csv("monster_com-job_sample.csv", parse_dates=["date_added"], on_bad_lines='skip')

# View first 10 rows
print(f"Dataset contains {df.shape[0]} rows and {df.shape[1]} columns.\n")
display(df.head(10))  # show first 10 rows for better feel
```

```
Dataset contains 22000 rows and 14 columns.
```

	country	country_code	date_added	has_expired	job_board	job_description	job_titl
0	United States of America	US	NaT	No	jobs.monster.com	TeamSoft is seeing an IT Support Specialist to...	IT Suppo Technician Jo in Madiso
1	United States of America	US	NaT	No	jobs.monster.com	The Wisconsin State Journal is seeking a flexi...	Busines Reporter/Edito Job in Madiso
2	United States of America	US	NaT	No	jobs.monster.com	Report this job About the Job DePuy Synthes Co...	Johnson Johnso Family c Companie Job Appl.
3	United States of America	US	NaT	No	jobs.monster.com	Why Join Altec? If you're considering a career...	Engineer Quality Job i Dixo
4	United States of America	US	NaT	No	jobs.monster.com	Position ID# 76162 # Positions 1 State CT C...	Shi Supervisor Part-Time Jo in Camph
5	United States of America	US	NaT	No	jobs.monster.com	Job Description Job #: 720298Apex Systems has...	Constructio PM Charlottesvill Job in Charl.
6	United States of America	US	NaT	No	jobs.monster.com	Report this job About the Job Based in San Fra...	CyberCoder Job Applicatio for Principa QA E.
7	United States of America	US	NaT	No	jobs.monster.com	RESPONSIBILITIES:Kforce has a client seeking a...	Mailroom Cler Job in Austi
8	United States of America	US	NaT	No	jobs.monster.com	Part-Time, 4:30 pm - 9:30 pm, Mon - Fri Brookd...	Housekeepe Job in Austi
9	United States of America	US	NaT	No	jobs.monster.com	Insituform Technologies, LLC, an Aegion compan...	Video Dat Managemer /Transportatio Technici.

```
#Dropping columns that have no unique values
cols_to_drop = ['country', 'country_code', 'has_expired', 'job_board']
drop_existing = [col for col in cols_to_drop if col in df.columns]
df.drop(columns=drop_existing, inplace=True)
print(f"Dropped columns: {drop_existing}")
```

```
Dropped columns: ['country', 'country_code', 'has_expired', 'job_board']
```

Drop missing rows in ['country', 'country_code', 'has_expired', 'job_board'] fields since if they are missing it won't help us achieve our goals outlined in the problem statement. For classification it will not matter if the job has expired, what country the country code and what job board the position is from. Since all the positions are from monster jobs and all are from 2016 and located within the USA these factors will not play any significant role in determining the job_title.

```
Start coding or generate with AI.
```

```
print("\nSummary statistics for categorical columns:\n")
display(df.describe(include=[object]))
```

```
Summary statistics for categorical columns:
```

	job_description	job_title	job_type	location	organization	page_url
count	22000	22000	20372	22000	15133	22000
unique	18744	18759	39	8423	738	22000
top	12N Horizontal Construction Engineers Job Desc...	Monster	Full Time	Dallas, TX	Healthcare Services	http://jobview.monster.com/Custodian-Lead-Job-...
freq	104	318	6757	646	1919	1

Findings:

- Total rows in data: 22000
- Features with only 1 unique entry: country, country_code, has_expired, job_board.
- These features will not be able to help us determine anything about the job market or trends due to there being a lack of unique values. These columns will later be dropped when creating models

```
# Column types and missingness summary
print("\nColumn types:")
display(df.dtypes)
```

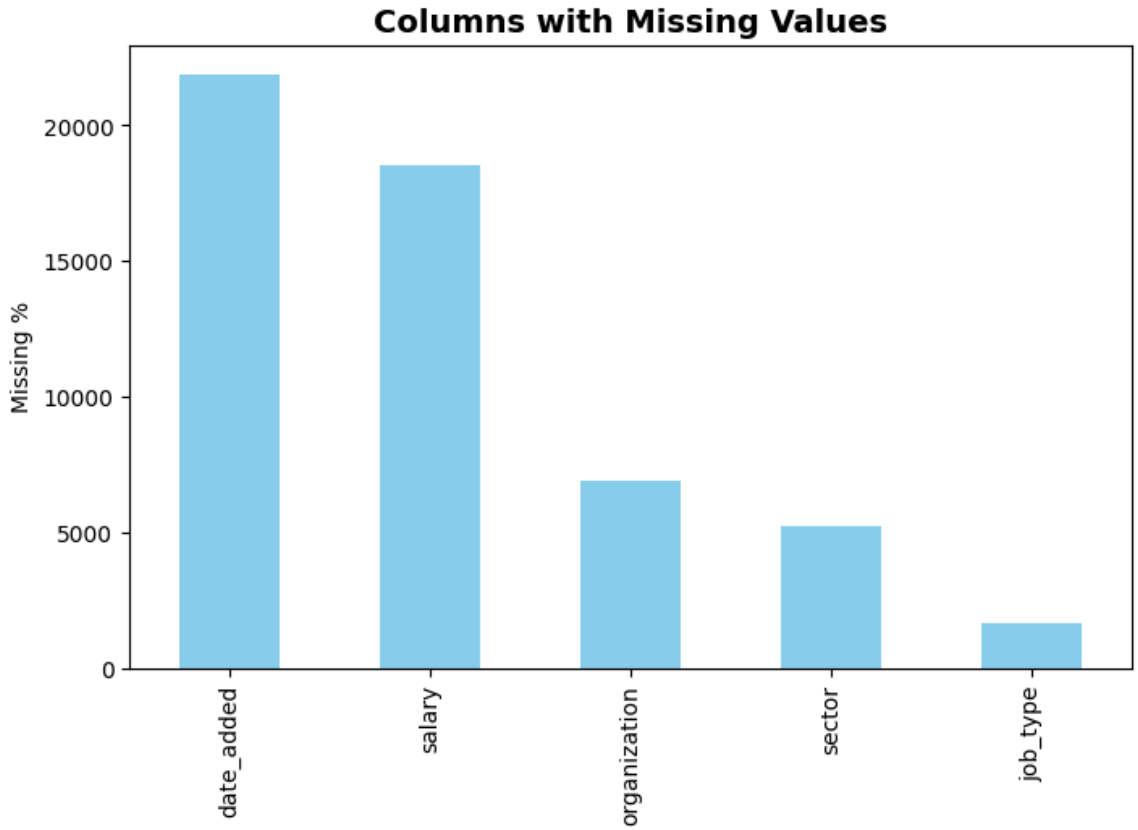
Column types:

	0
date_added	datetime64[ns]
job_description	object
job_title	object
job_type	object
location	object
organization	object
page_url	object
salary	object
sector	object
uniq_id	object

dtype: object

```
#Missing values analysis
missing = df.isnull().sum().sort_values(ascending=False)
missing_nonzero = missing[missing > 0]
print("Columns with missing values:", len(missing_nonzero), "/", df.shape[1])

# Optional: Visualize missingness
plt.figure(figsize=(8, 5))
missing_nonzero.head(20).plot(kind='bar', color='skyblue')
plt.ylabel("Missing %")
plt.title("Columns with Missing Values",fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.show()
```

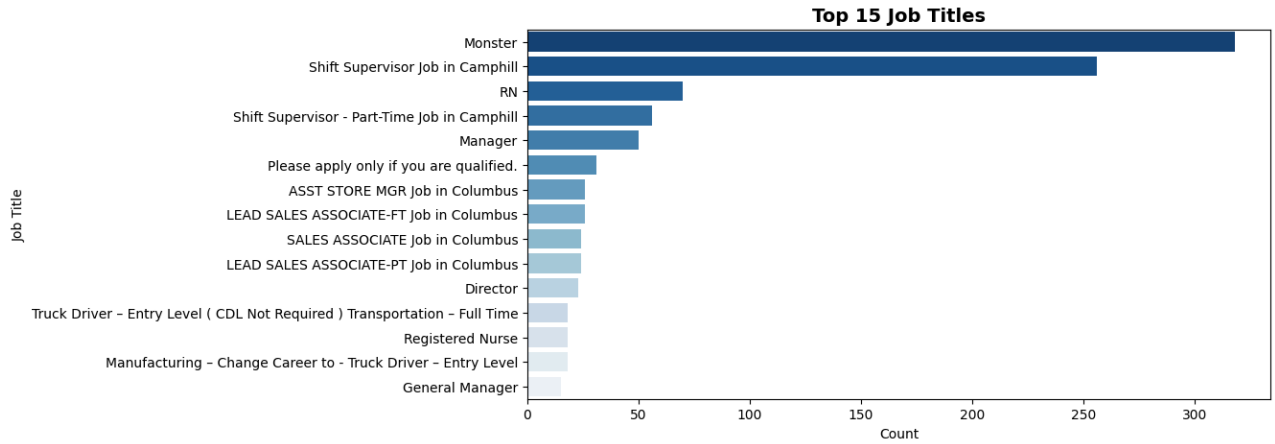
```
Columns with missing values: 5 / 10
```



Findings:

- After understanding the data types, and percentage of data missing in each field, continue to inspect the data and prepare to clean and adjust data fields
- Based on the results above regarding the percentage of data missing in each field, the top missing features in the data are date_added, salary, and organization. -The date_added column is not signifcant to this problem as we are not worried about when the job was posted we know this data was from the year 2016

```
# Basic counts for key fields
# After
top_jobs = df['job_title'].value_counts().head(15)
plt.figure(figsize=(10,5))
sns.barplot(x=top_jobs.values, y=top_jobs.index, palette="Blues_r", hue = top_jobs.index, legend=False
plt.title("Top 15 Job Titles",fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.xlabel("Count")
plt.ylabel("Job Title")
plt.show()


print("\nTop 10 companies:")
display(df['organization'].value_counts().head(10))
```



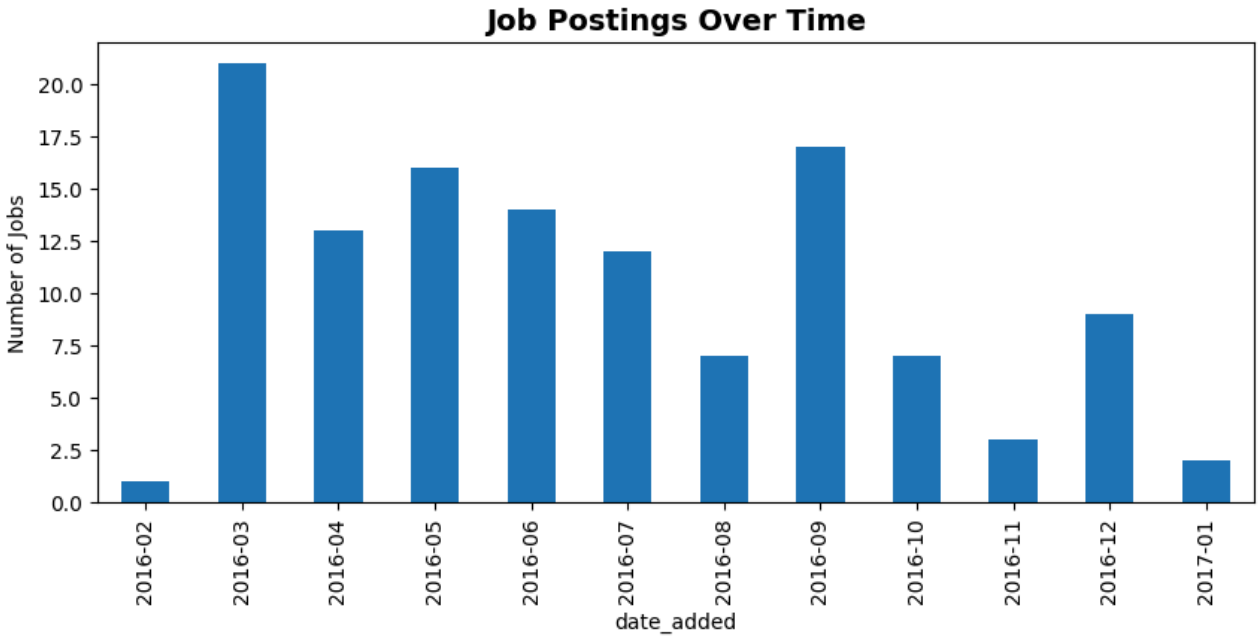
Top 10 companies:

organization	count
Healthcare Services	1919
All	1158
Retail	1081
Other/Not Classified	1048
Manufacturing - Other	885
Computer/IT Services	822
Legal Services	466
Business Services - Other	410
Restaurant/Food Services	384
Transport and Storage - Materials	342

dtype: int64

```
# If posted_date exists: show date range
if 'date_added' in df.columns:
    print("\nPosted date range:", df['date_added'].min(), "to", df['date_added'].max())

    plt.figure(figsize=(10,4))
    df['date_added'].dt.to_period('M').value_counts().sort_index().plot(kind='bar')
    plt.title("Job Postings Over Time",fontdict={'fontsize': 14, 'fontweight': 'bold'})
    plt.ylabel("Number of Jobs")
    plt.show()
```

```
Posted date range: 2016-02-29 00:00:00 to 2017-01-16 00:00:00
```



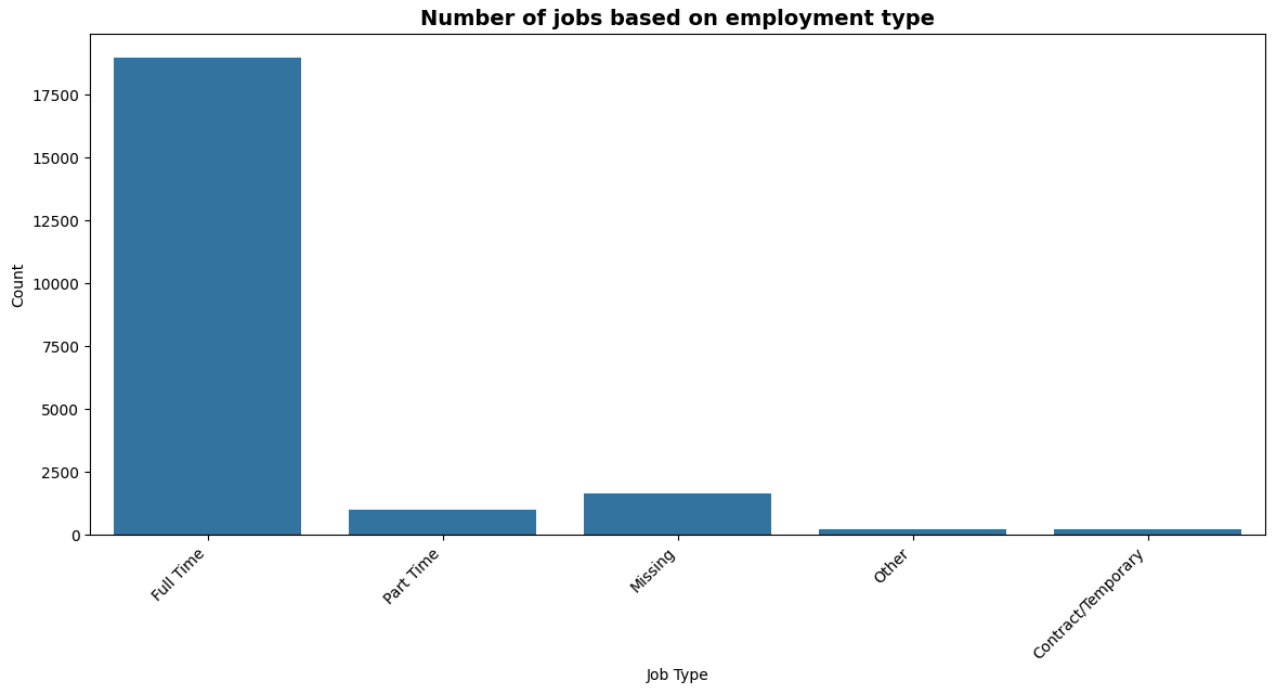
Findings:

- Record percent missing for salary and any non-standard formats observed (ranges, hourly rates).
- Note the top job titles and companies (high cardinality fields that may need aggregation).
- Not many jobs posted have dates will have to focus more towards the job_title and job_description fields (categorical problem).

```
#filter the job types and clump any with full time or part time
def clean_job_type(job_type):
    """
    Standardizes job type strings into consistent categories.
    """
    if pd.isna(job_type) or job_type.strip() == '':
        return 'Missing'

    # Convert to lowercase for case-insensitive matching
    jt_lower = job_type.lower()

    if 'full time' in jt_lower or 'full-time' in jt_lower:
        # Check for hybrid types and prioritize 'Full Time' if it's the dominant part
        if 'part time' in jt_lower:
             # Often these mixed roles are primarily full-time but flexible, we'll categorize them as
            return 'Full Time'
        return 'Full Time'

    elif 'part time' in jt_lower or 'part-time' in jt_lower:
        return 'Part Time'

    elif 'contract' in jt_lower or 'temporary' in jt_lower or 'temp' in jt_lower or 'seasonal' in jt_l
        return 'Contract/Temporary'

    # Catch any remaining unique values that might not fit the main categories
    return 'Other'

# --- 2. Apply the Cleaning Function ---
```

```
# Apply the function to create a new, clean job type column
df['job_type_clean'] = df['job_type'].apply(clean_job_type)
```

```
#Create plot based on employment types
plt.figure(figsize=(14,6))
sns.countplot(x='job_type_clean', data=df)
plt.title('Number of jobs based on employment type', fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.xlabel('Job Type')
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.ylabel('Count')

plt.show()
```



Findings:

- Majority of the jobs being posted are Full Time positions with a small portion of positions being temporary or part time.

Filtering Location to get Region

```
#take location and extract the state to create region feature
locations = df['location']
state_location = []
for x in range(len(locations)):

        if locations.get(x) != None:
                location = locations.get(x)
                location = str(location)
                area = location.split(',')
                state = area[len(area)-1]
                if(state[1:3].isupper()):
                        state_location.append(state[1:3])
                else:
```

```
                        state_location.append('N/A')
        else:
                state_location.append('N/A')
```

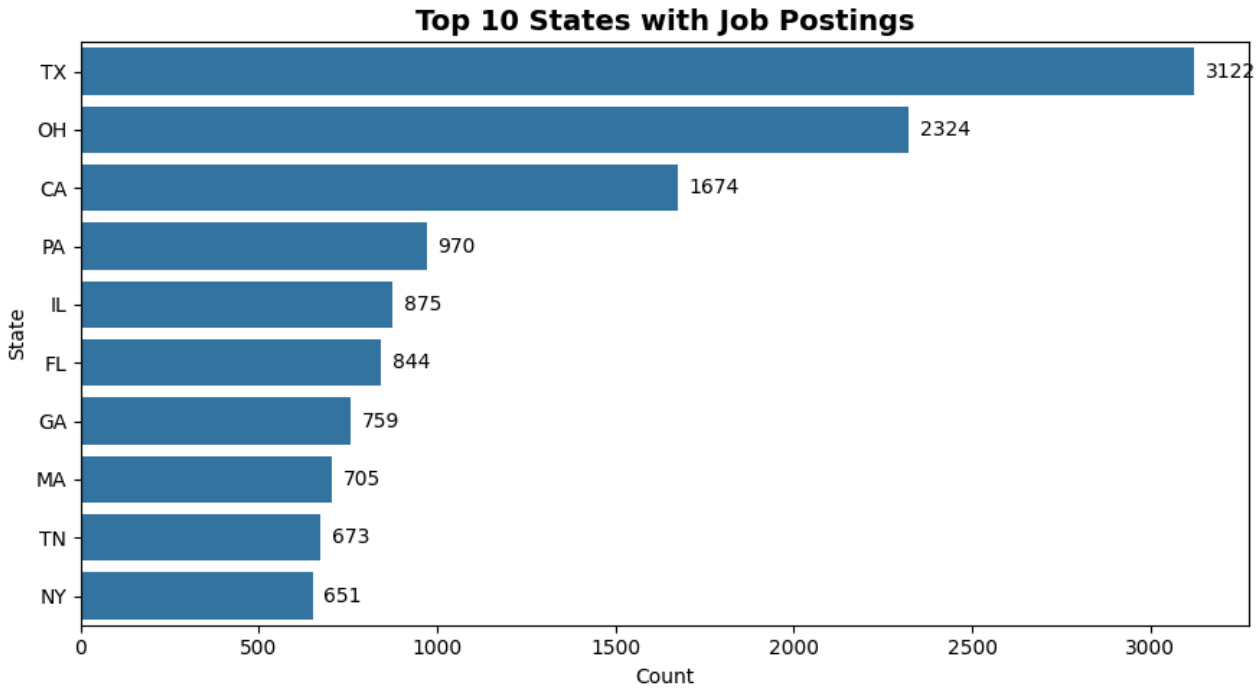
```
#create the state feature in the dataframe
df['state']= state_location
```

```
#Classify the states into their respective regions
state_regions = {
    'AL': 'South', 'AK': 'West', 'AZ': 'West', 'AR': 'South', 'CA': 'West Coast',
    'CO': 'West', 'CT': 'Northeast', 'DE': 'South', 'FL': 'South', 'GA': 'South',
    'HI': 'West', 'ID': 'West', 'IL': 'Midwest', 'IN': 'Midwest', 'IA': 'Midwest',
    'KS': 'Midwest', 'KY': 'South', 'LA': 'South', 'ME': 'Northeast', 'MD': 'South',
    'MA': 'Northeast', 'MI': 'Midwest', 'MN': 'Midwest', 'MS': 'South', 'MO': 'Midwest',
    'MT': 'West', 'NE': 'Midwest', 'NV': 'West', 'NH': 'Northeast', 'NJ': 'Northeast',
    'NM': 'West', 'NY': 'Northeast', 'NC': 'South', 'ND': 'Midwest', 'OH': 'Midwest',
    'OK': 'South', 'OR': 'West Coast', 'PA': 'Northeast', 'RI': 'Northeast',
    'SC': 'South', 'SD': 'Midwest', 'TN': 'South', 'TX': 'South', 'UT': 'West',
    'VT': 'Northeast', 'VA': 'South', 'WA': 'West Coast', 'WV': 'South',
    'WI': 'Midwest', 'WY': 'West', 'DC': 'Northeast'
}
```

```
# Strip whitespace and make uppercase just in case
import re

# Extract the 2-letter state abbreviation using regex
df['state'] = df['location'].str.extract(r',\s*([A-Z]{2})\s*\d{0,5}')


# Map states to regions
df['region'] = df['state'].map(state_regions)
```

```
import re

def split_location(loc):
    if pd.isna(loc):
        return pd.Series([None, None])
    s = str(loc).strip()
    # If there's a comma, split as "City, ST"
    if "," in s:
        city, rest = s.split(",", 1)
        state_raw = rest.strip()
    else:
        city, state_raw = None, s  # handle formats like "TX 75201"
    # Extract 2-letter state code
    m = re.search(r"\b([A-Za-z]{2})\b", str(state_raw).upper())
    state = m.group(1).upper() if m else None
    city = None if city is None else city.strip()
    return pd.Series([city, state])

df[['city','state']] = df['location'].astype(str).apply(split_location)

def normalize_state(x):
    if pd.isna(x):
        return None
    x = str(x).strip().upper()
    m = re.match(r"^([A-Z]{2})", x)
    return m.group(1) if m else None

df['state'] = df['state'].apply(normalize_state)
```

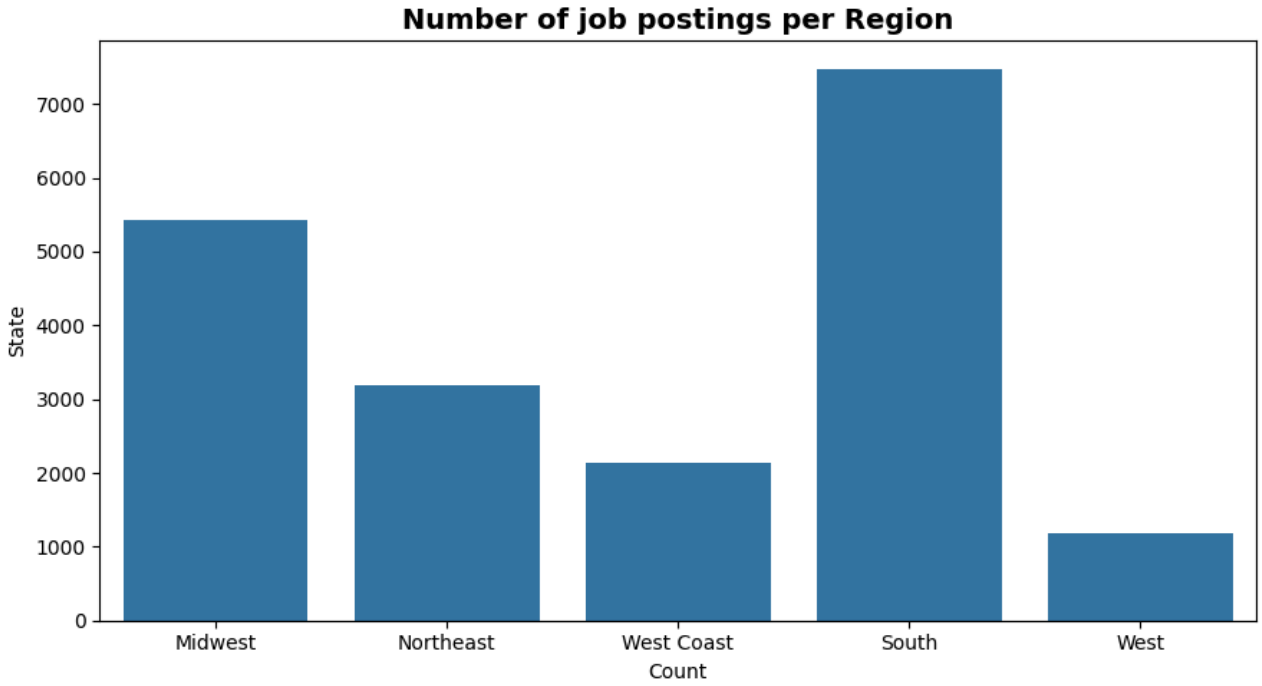
```
valid_states = set("""AL AK AZ AR CA CO CT DE FL GA HI ID IL IN IA KS KY LA ME MD MA MI MN
MS MO MT NE NV NH NJ NM NY NC ND OH OK OR PA RI SC SD TN TX UT VT VA WA WV WI WY DC""".split())
df['state_clean'] = df['state'].where(df['state'].isin(valid_states))
top_states = df['state_clean'].value_counts().head(10)

plt.figure(figsize=(9, 5))
ax = sns.barplot(x=top_states.values, y=top_states.index)
ax.set_title("Top 10 States with Job Postings", fontsize=14, weight="bold")
ax.set_xlabel("Count")
ax.set_ylabel("State")

# Add value labels
for i, v in enumerate(top_states.values):
    ax.text(v + max(top_states.values)*0.01, i, str(int(v)), va='center')

plt.tight_layout()
plt.show()
```



Findings:

- Texas has the most job postings
- Location field now split up to gain a better insight into the data

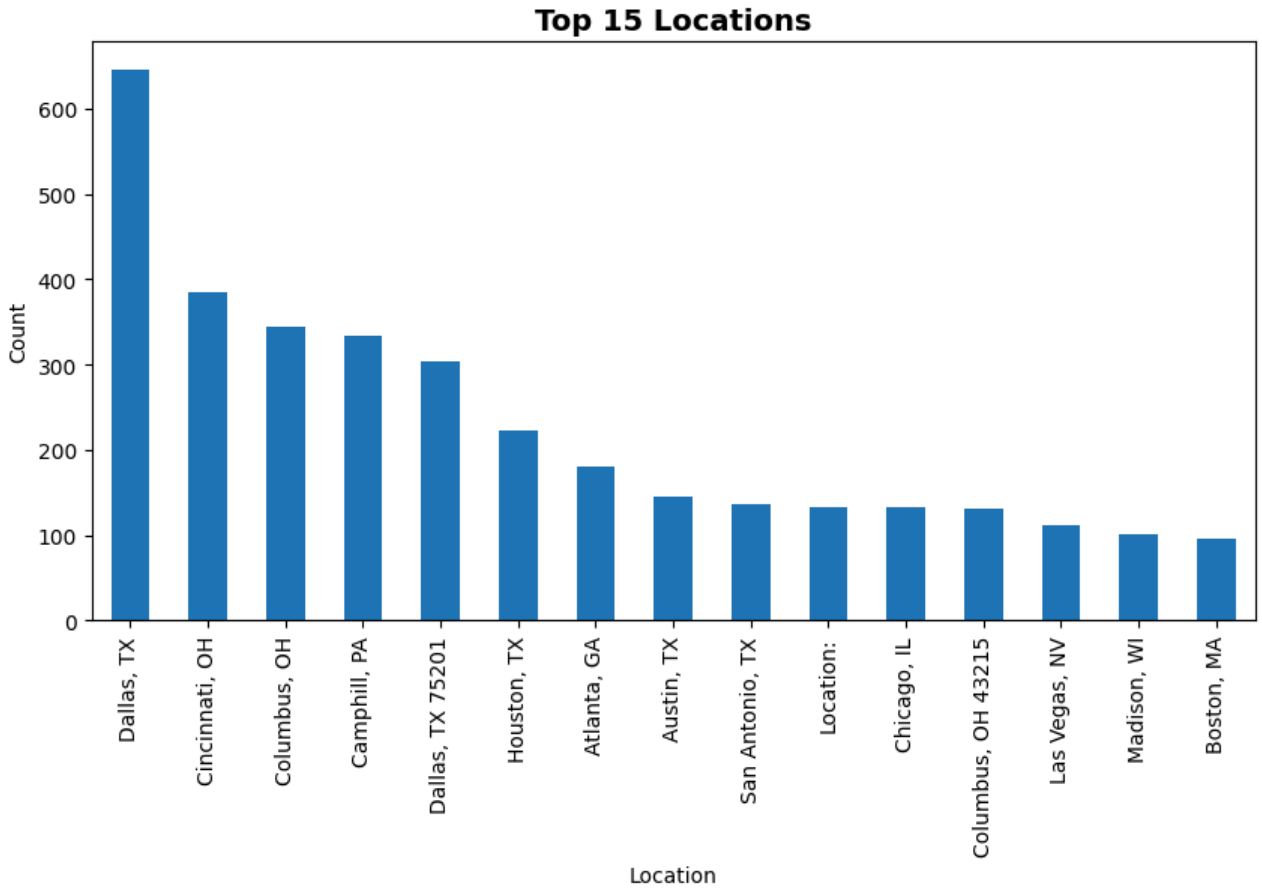
```
#Number of jobs based on region
plt.figure(figsize=(9, 5))
ax = sns.countplot(x='region', data = df)
ax.set_title("Number of job postings per Region", fontsize=14, weight="bold")
ax.set_xlabel("Count")
ax.set_ylabel("State")
plt.tight_layout()
plt.show()
```



Findings:

- Most jobs are located within the South and Midwest
- This could mean the Monster website is a primary source for employing in these regions
- Could also mean that these locations are growing and have more openings than the other regions

```
#Top locations posted
plt.figure(figsize=(10,5))
df['location'].value_counts().head(15).plot(kind='bar')
plt.title("Top 15 Locations", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.xlabel("Location")
plt.ylabel("Count")
plt.show()
```



Findings:

- Dallas is the city with the most job postings can notice that some of the cities are major cities, dallas, houstan, atlanta, las vegas, boston etc.
- Large cities have more companies looking to hire and wanting to expand.

Filtering job_titles into job_category

I want to change this to check skills

```
#Taking the job_titles and extracting the field ex. tech, healthcare
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

def lemmatize_title(title):
    words = word_tokenize(str(title).lower())
    lemmas = [lemmatizer.lemmatize(word) for word in words]
    return ' '.join(lemmas)
```

```
#Extract specific titles and create a new field lemmatized_title
df['lemmatized_title'] = df['job_title'].apply(lemmatize_title)
```

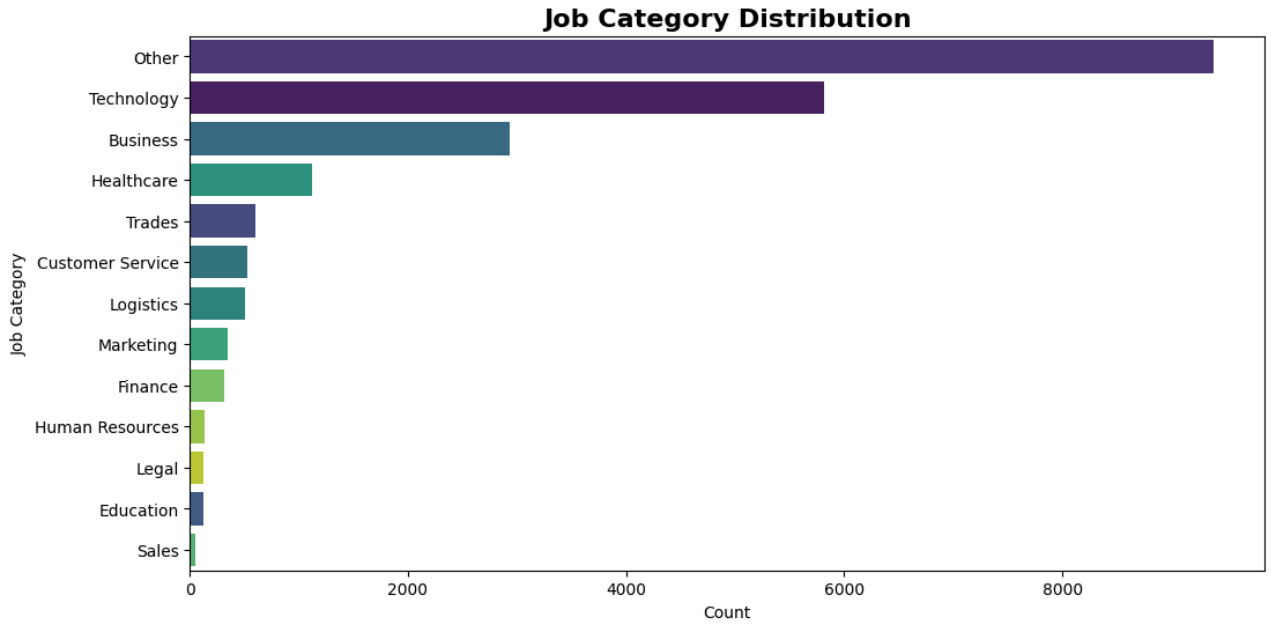
```
#create job categories can be changed in future to expand for other jobs that might not be captured
job_categories = {
    'developer': 'Technology',
    'software': 'Technology',
    'engineer': 'Technology',
    'it': 'Technology',
    'data': 'Technology',
    'nurse': 'Healthcare',
    'health': 'Healthcare',
    'care taker': 'Healthcare',
    'doctor': 'Healthcare',
    'pharmacist': 'Healthcare',
    'physician': 'Healthcare',
    'hospital': 'Healthcare',
    'medical': 'Healthcare',
    'teacher': 'Education',
    'education': 'Education',
    'school': 'Education',
    'student': 'Education',
    'sales': 'Sales',
    'marketing': 'Marketing',
    'professor': 'Education',
    'accountant': 'Finance',
    'finance': 'Finance',
    'analyst': 'Business',
    'manager': 'Business',
    'sales': 'Sales',
    'marketing': 'Marketing',
    'customer': 'Customer Service',
    'support': 'Customer Service',
    'warehouse': 'Logistics',
    'driver': 'Logistics',
    'mechanic': 'Trades',
    'construction': 'Trades',
    'electrician': 'Trades',
    'plumber': 'Trades',
    'recruiter': 'Human Resources',
    'hr': 'Human Resources',
    'lawyer': 'Legal',
    'attorney': 'Legal'
}
```

```
#mapping the lemmatized_title to job_category
def map_category(title):
    title = str(title).lower()
    for keyword, category in job_categories.items():
        if keyword in title:
            return category
    return 'Other'  # Default if no keyword matched

df['job_category'] = df['lemmatized_title'].apply(map_category)
```

Mapping the job_title field into job_category to get a better idea of what industries are looking to hire based on the title of the job

```
#plotting the count of job categories
plt.figure(figsize=(12,6))
sns.countplot(data=df, y='job_category', order=df['job_category'].value_counts().index, palette='viric
plt.title('Job Category Distribution', fontdict={'fontsize': 16, 'fontweight': 'bold'} )
```

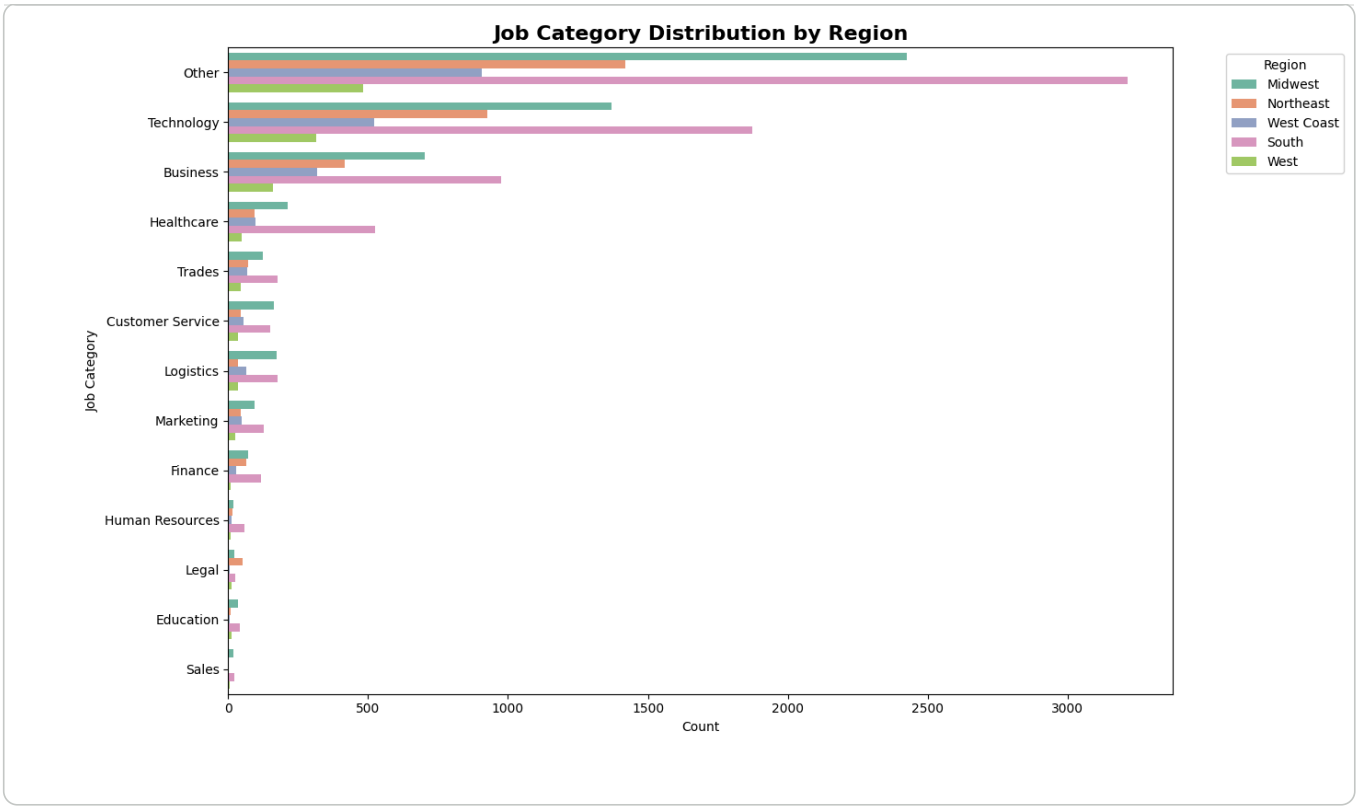
```
plt.xlabel('Count')
plt.ylabel('Job Category')
plt.show()
```



Findings:

- Besides other, Technology and Business have the most job postings with sales and education having the least
- Reasons could be that previous years hired more Education workers and sales workers and no longer need as many, could also be an indication of industry trends.
- As technology evolves, new business are able to form creating an increase for both tech and business workers.
- Question: How long will these growing/booming industries last?

```
#number of jobs in each region based upon the job field
plt.figure(figsize=(14,8))
sns.countplot(data=df, y='job_category', hue='region', order=df['job_category'].value_counts().index,
plt.title('Job Category Distribution by Region', fontdict={'fontsize': 16, 'fontweight': 'bold'})
plt.xlabel('Count')
plt.ylabel('Job Category')
plt.legend(title='Region', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



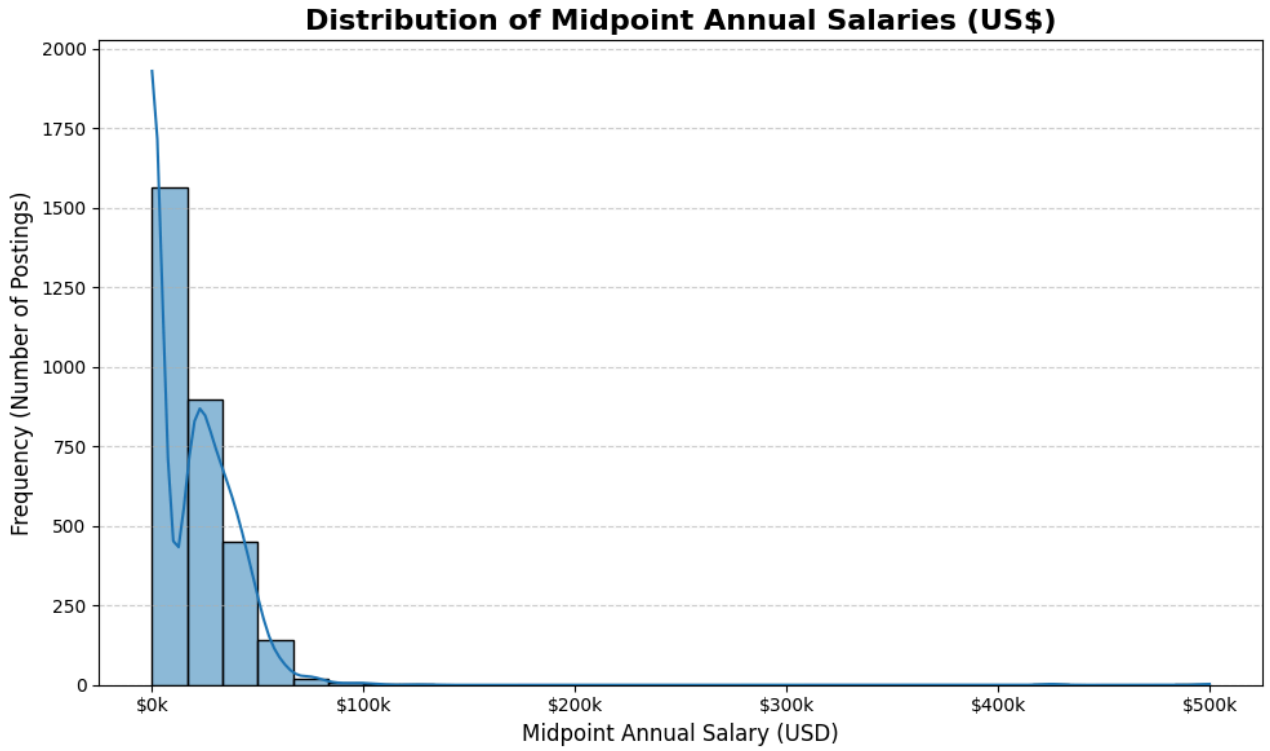
Findings:

- While the Southern region has most of the job postings, the Midwestern region has more positions within the customer service, trades, logistics and sales industries.
- This can indicate a growth/developement in different regions. For example, the northeast is very established within the Tech and business fields, so they may not need to hire as many people as a growing or developing region such as the South.
- As a region develops, more people need to be hired to fill positions. Reasons a region may be developing could be from cost of business operating in other states. For example, cost of operating in California is more expensive than the cost of operating in Texas.

Cleaning Salary and calculating the midpoint

```
def extract_midpoint_salary(salary_str):
    """
    Cleans the 'salary' string to extract the numerical range and calculates the midpoint.
    Returns 0 for missing/unclean values.
    """
    if pd.isna(salary_str):
        return None

    # 1. Standardize and remove common symbols ($, comma, /year, etc.)
    cleaned_str = salary_str.lower().replace(',', '').replace('$', '').replace('/year', '').strip()

    # 2. Extract all numbers (min/max)
    numbers = re.findall(r'\d+', cleaned_str)

    if not numbers:
        return None

    # 3. Handle single number (only max) or range (min and max)
    if len(numbers) >= 2:
```

```
            # Range available: Use the average (midpoint) of the first two numbers found
            min_sal = int(numbers[0])
            max_sal = int(numbers[1])
            return (min_sal + max_sal) / 2
        elif len(numbers) == 1:
            # Only one number available (assume it's the target salary or a starting point)
            return int(numbers[0])

        return None

    # Apply the function to the original 'salary' column
    df['salary'].fillna(0)
    df['midpoint_salary'] = df['salary'].apply(extract_midpoint_salary)

    # Filter out rows where salary could not be calculated
    df_salary_clean = df.dropna(subset=['midpoint_salary']).copy()

    # --- 2. Plotting the Distribution (Histogram) ---

    plt.figure(figsize=(10, 6))

    # Use a Histogram to show the distribution of the continuous salary data
    sns.histplot(
        data=df_salary_clean,
        x='midpoint_salary',
        bins=30, # Number of bins (ranges) for the salary
        kde=True, # Add a kernel density estimate curve
        color='#1f77b4'
    )

    plt.title('Distribution of Midpoint Annual Salaries (US$)',
              fontdict={'fontsize': 16, 'fontweight': 'bold'})
    plt.xlabel('Midpoint Annual Salary (USD)', fontsize=12)
    plt.ylabel('Frequency (Number of Postings)', fontsize=12)

    # Format x-axis labels to look like currency
    from matplotlib.ticker import FuncFormatter
    formatter = FuncFormatter(lambda x, pos: f'${int(x/1000)}k')
    plt.gca().xaxis.set_major_formatter(formatter)

    plt.grid(axis='y', linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.show()

    print(f"Total postings with usable annual salary data: {len(df_salary_clean)}")
```

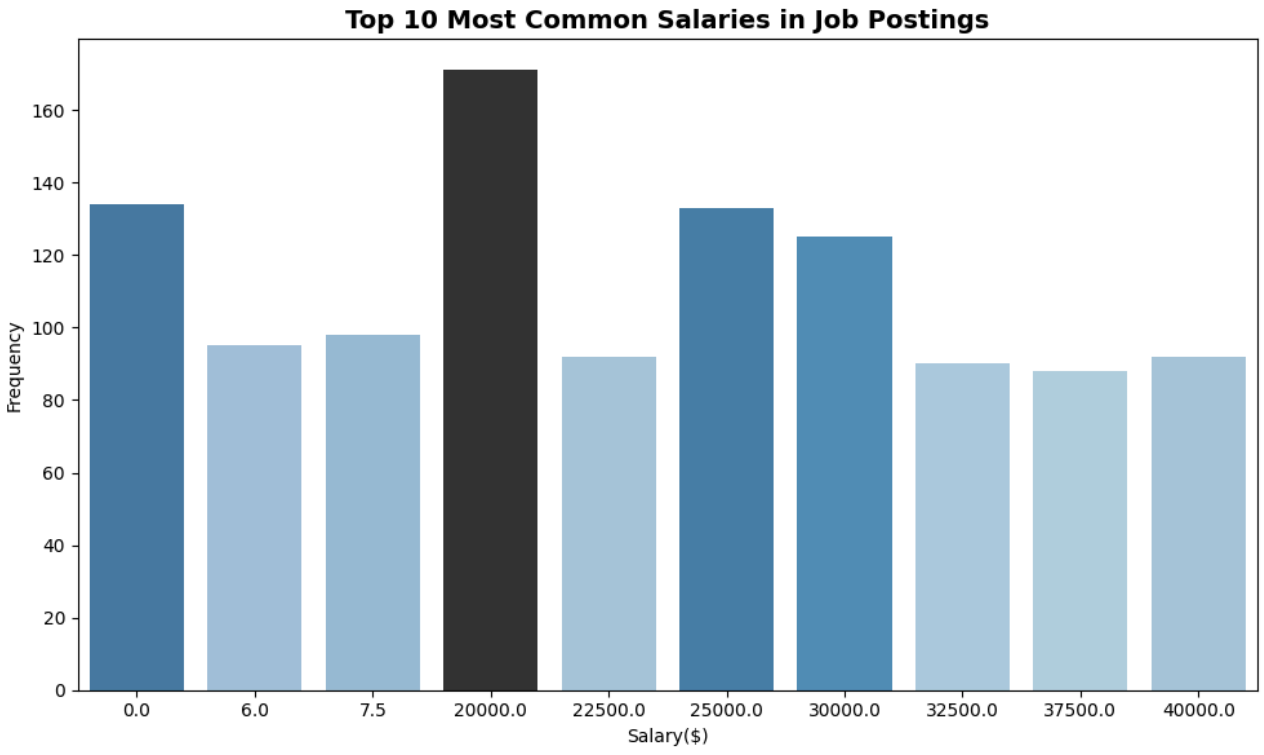



```
Total postings with usable annual salary data: 3085
```

Findings:

- There are some positions with no listed salary, this could be unpaid positions, such as unpaid internships or these could be posts that did not include the salary/pay.
- Positions listed with low salaries could indicate positions with a percentage of sales which would not be included in this information such as a car salesman.
- This could also give us insight into minimum wages and pay across various states/regions and pay depending on indemand skills.

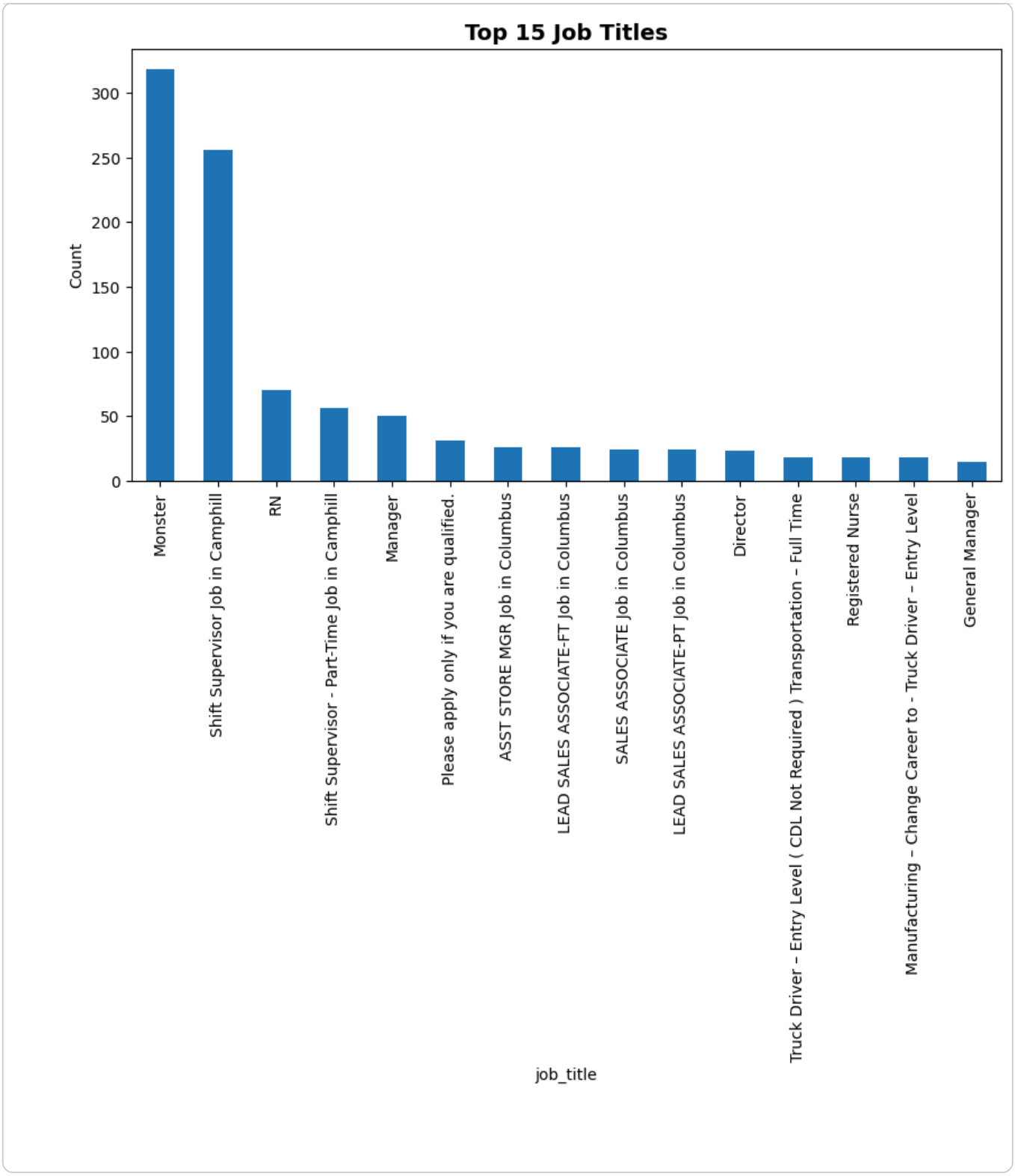
```
#visualize the top salaries
top_salaries = df_salary_clean['midpoint_salary'].value_counts().nlargest(10)

# Step 4: Plot
plt.figure(figsize=(10, 6))
sns.barplot(x=top_salaries.index, y=top_salaries, palette="Blues_d", hue = top_salaries, legend=False)
plt.xlabel("Salary($)")
plt.ylabel("Frequency")
plt.title("Top 10 Most Common Salaries in Job Postings", fontdict={'fontsize': 14, 'fontweight': 'bold
plt.tight_layout()
plt.show()
```



Findings:

- Based on this visualization there is a spike in jobs at 100k salary with 140 people, and the next most common salary being 60k, this can be an indication that the positions require certain experience and skill sets that pay more
- This could also be an indication of inflation or the cost of living is increasing in the locations where these jobs were posted.

```
plt.figure(figsize=(10,5))
df['job_title'].value_counts().head(15).plot(kind='bar')
plt.title("Top 15 Job Titles", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.ylabel("Count")
plt.show()
```

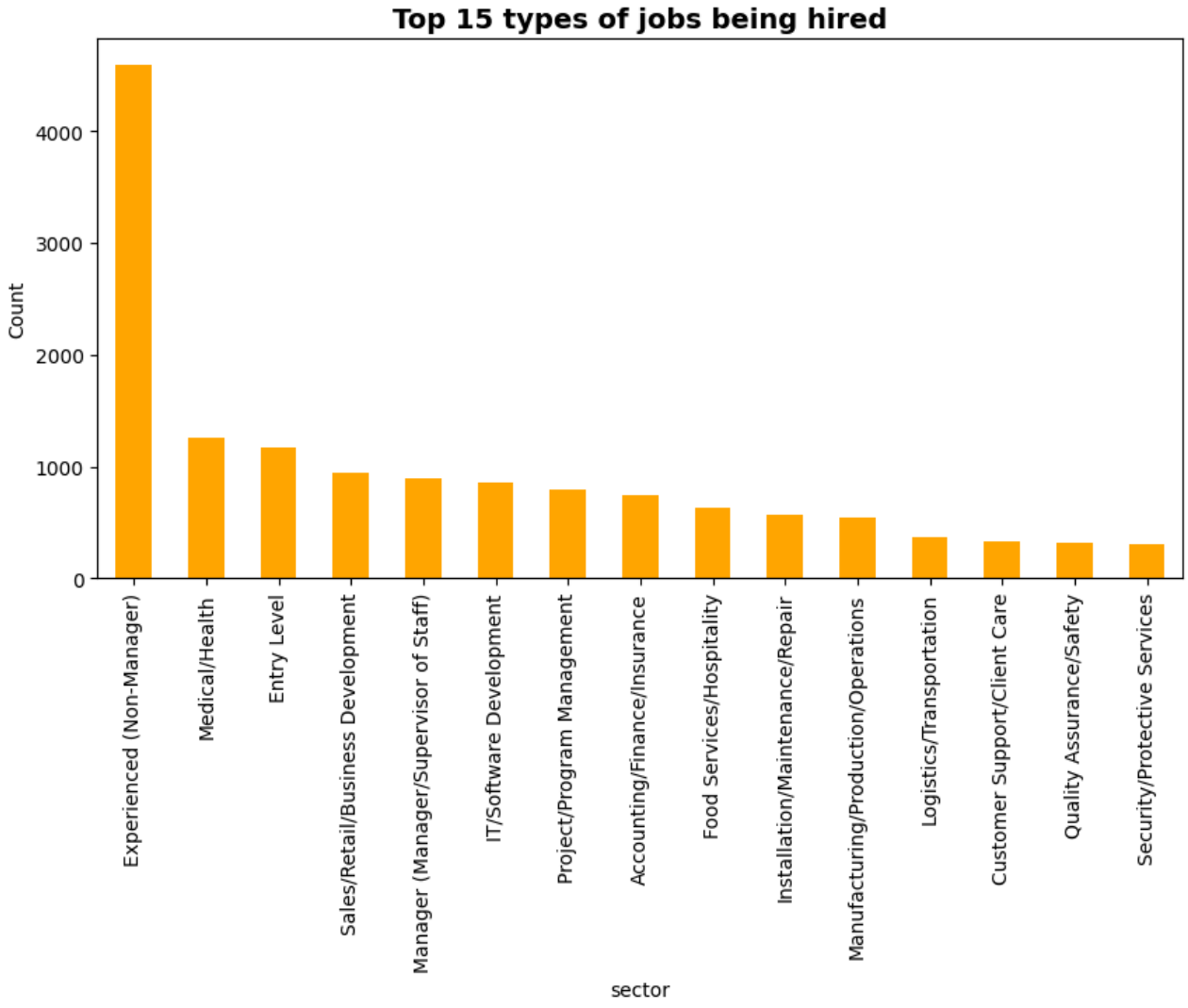


Findings:

- Employers did not correctly fill out the job posting since most jobs are with the title Monter.
- This could result in bias when trying to use the job_title to predict another field.

```
plt.figure(figsize=(10,5))
df['sector'].value_counts().head(15).plot(kind='bar', color='orange')
plt.title("Top 15 types of jobs being hired", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.ylabel("Count")
```

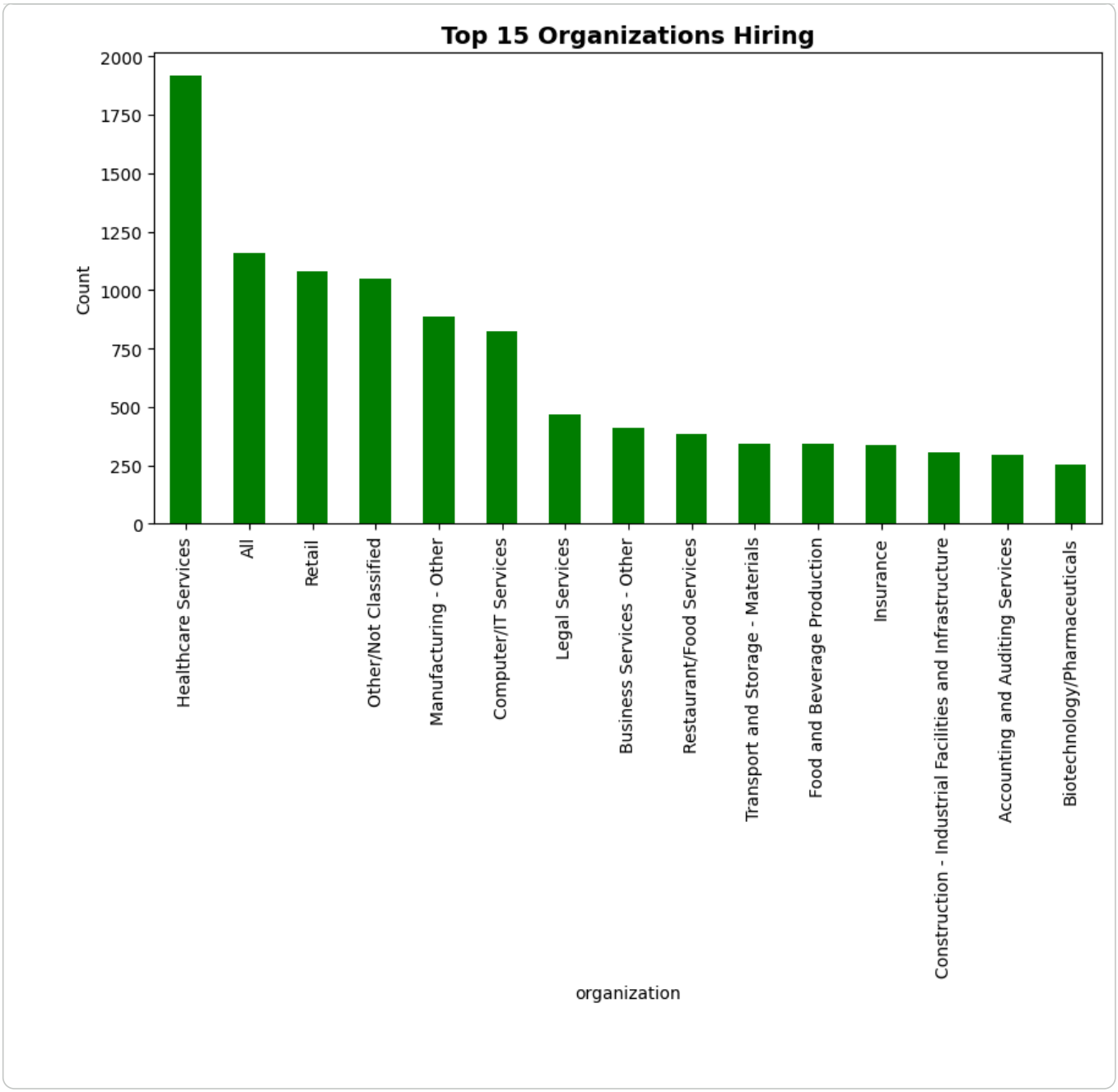
```
plt.show()
```



Findings:

- Most of the positions are for experienced workers, not as many entry/college-grad postions.
- Other industries seem to have a decent amount of postings
- Data is skewed heavily towards experienced workers

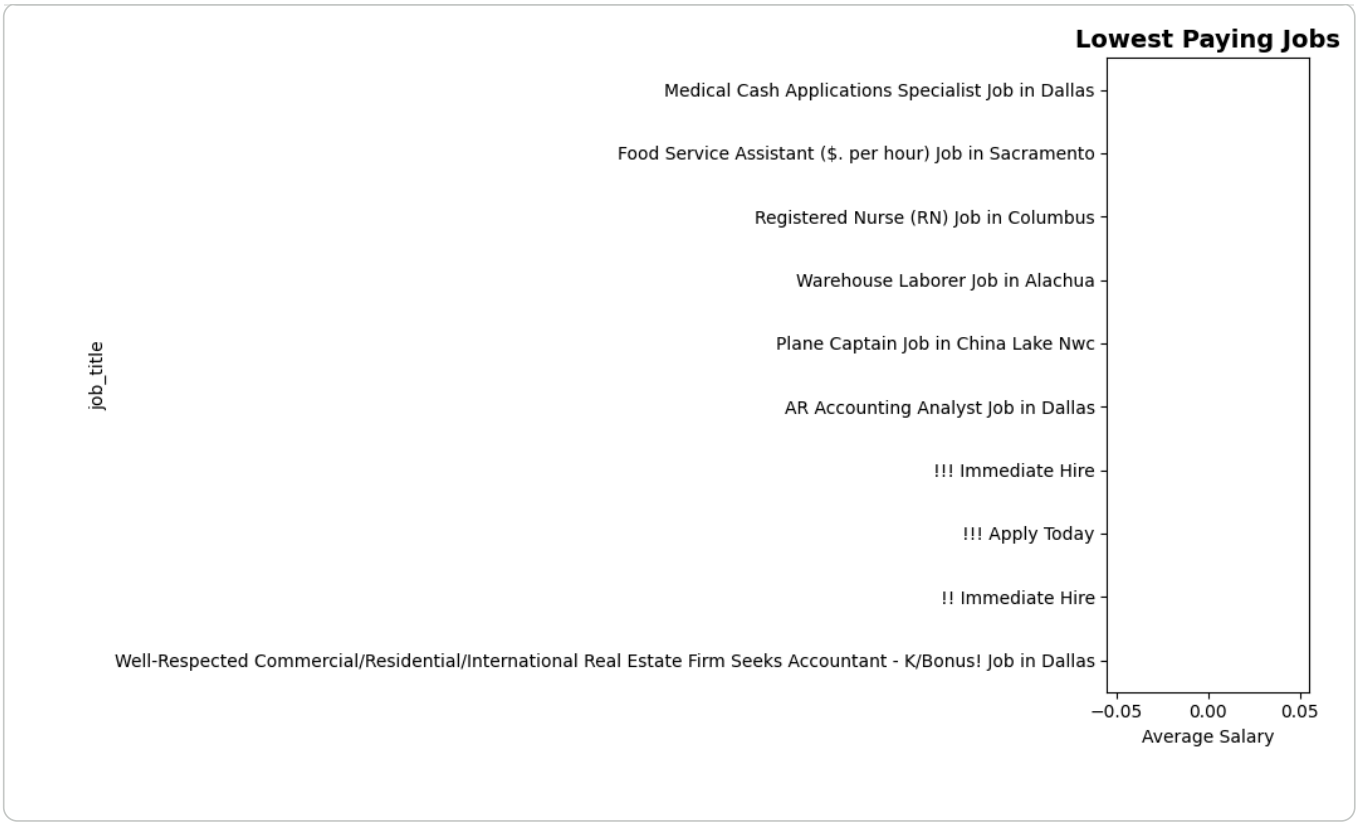
```
plt.figure(figsize=(10,5))
df['organization'].value_counts().head(15).plot(kind='bar', color='green')
plt.title("Top 15 Organizations Hiring", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.ylabel("Count")
plt.show()
```



Findings:

- Seems as if companies have confused organization and sector and filled out the same information, this can also be from how the data was gathered.
- Health industry seems to be hiring the most people, could be an indemand field and can be due to people living longer

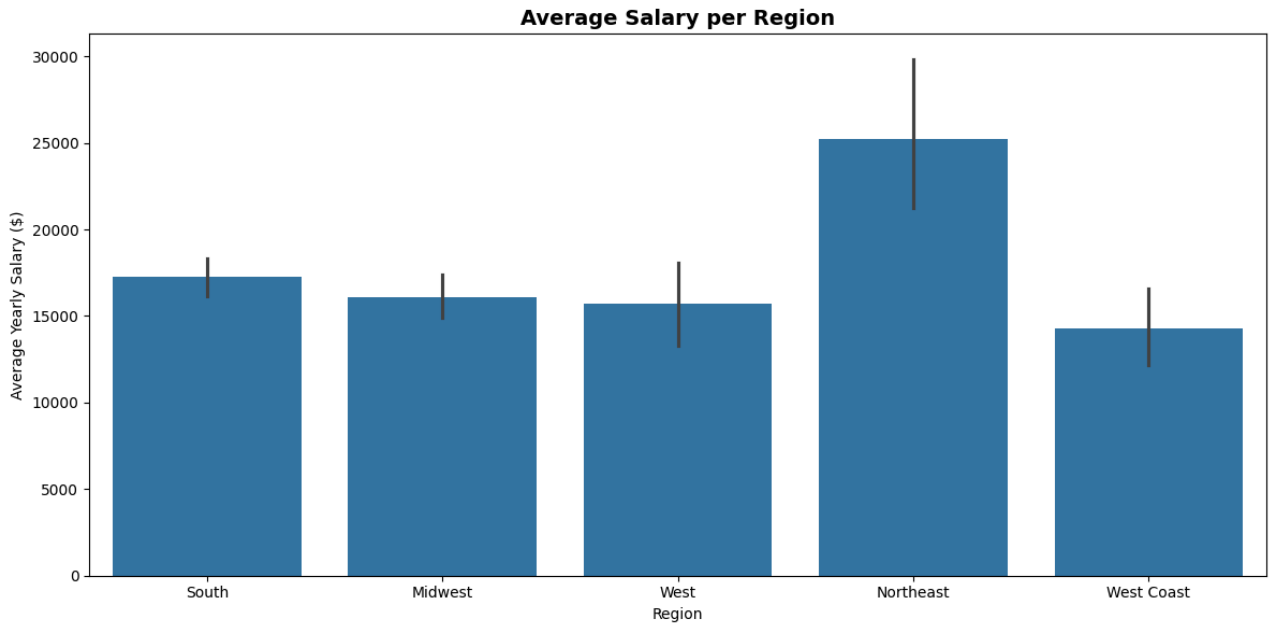
```
top_industries = df_salary_clean.groupby('job_title')['midpoint_salary'].min().sort_values(ascending=1
top_industries.plot(kind='barh', figsize=(10,6), color='teal')
plt.title("Lowest Paying Jobs", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.xlabel("Average Salary")
plt.tight_layout()
plt.show()
```



Findings:

- The lowest paying jobs don't seem to have any correlation with specific job markets or industries. However, a few are listed as being located in Dallas.
- These jobs could have been posted incorrectly, or may have the pay listed in the job description only.

```
plt.figure(figsize=(12, 6))
sns.barplot(data=df_salary_clean, x='region', y='midpoint_salary')
plt.xlabel("Region")
plt.ylabel("Average Yearly Salary ($)")
plt.title("Average Salary per Region", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.tight_layout()
plt.show()
```

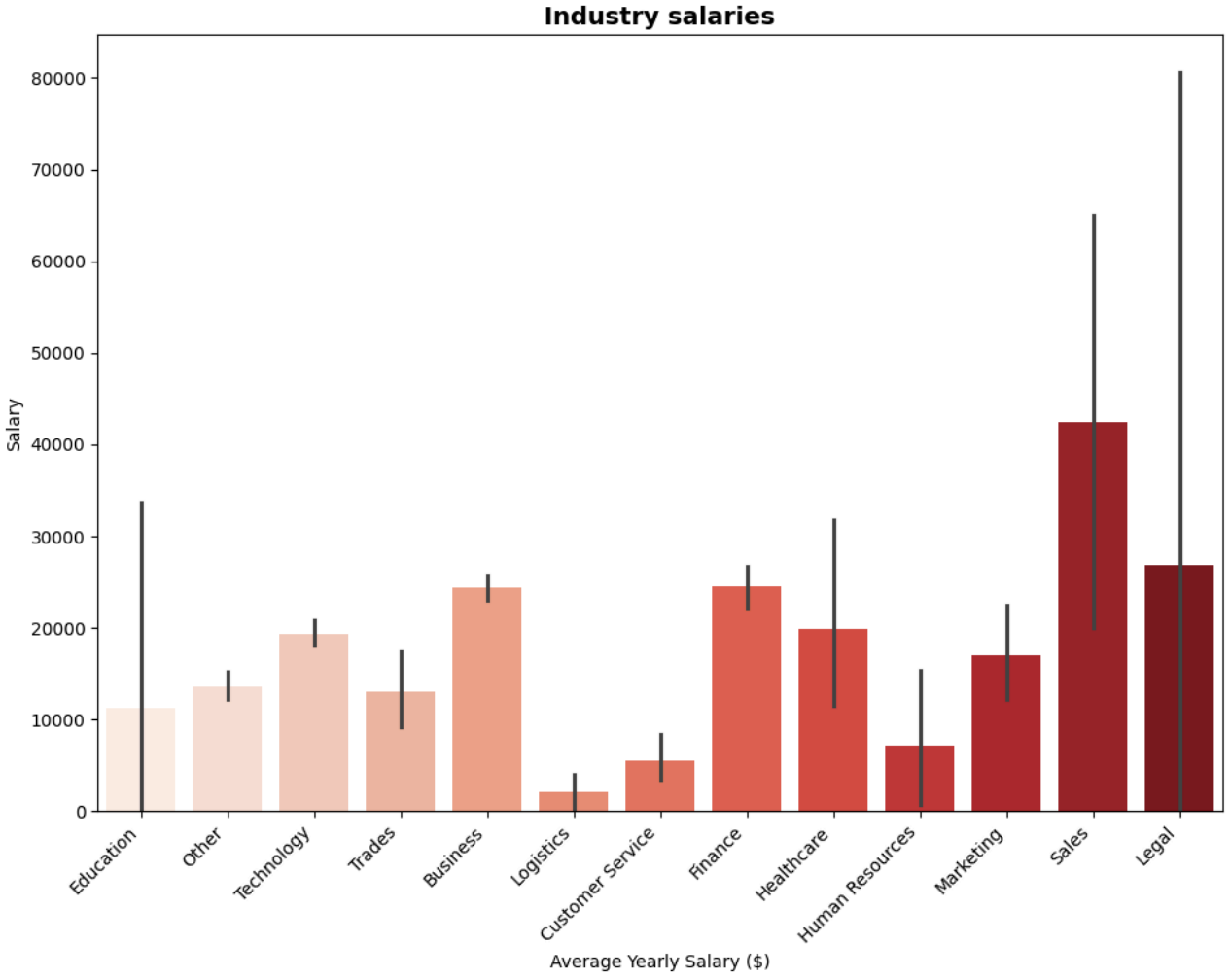


Findings:

- Northeast has the highest average salaries ranging to an extent of 30k for positions
- Majority of regions have salaries averaging 15k-20k
- This may not be an accurate account of salaries, as areas within the West Coast and Northeast being expensive to live in causing for an increased salary to compensate.

```
plt.figure(figsize=(10, 8))
sns.barplot(data=df_salary_clean, x='job_category', y='midpoint_salary', palette='Reds', legend= False
plt.xlabel("Average Yearly Salary ($)")
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.ylabel("Salary")
plt.title("Industry salaries", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.tight_layout()
plt.show()
```

```
/tmp/ipython-input-4139294674.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x`

  sns.barplot(data=df_salary_clean, x='job_category', y='midpoint_salary', palette='Reds', legend= Fals
```



Findings:

- Besides Other which may include various other industries, Sales has the highest Average Salary being greater than 40k.
- Legal has the highest recorded salary pay amount.
- This also can be inaccurate data from the dataset not having salaries recorded, a lot of industries such as tech have very high salaries and these amounts shown are very low for industry standards.

```
word_freq = Counter(" ".join(df['job_title'].dropna()).lower().split())
wordcloud = WordCloud(width=800, height=400, background_color='white').generate_from_frequencies(word_

plt.figure(figsize=(15, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Most Common Words in Job Titles", fontdict={'fontsize': 14, 'fontweight': 'bold'})
plt.show()
```

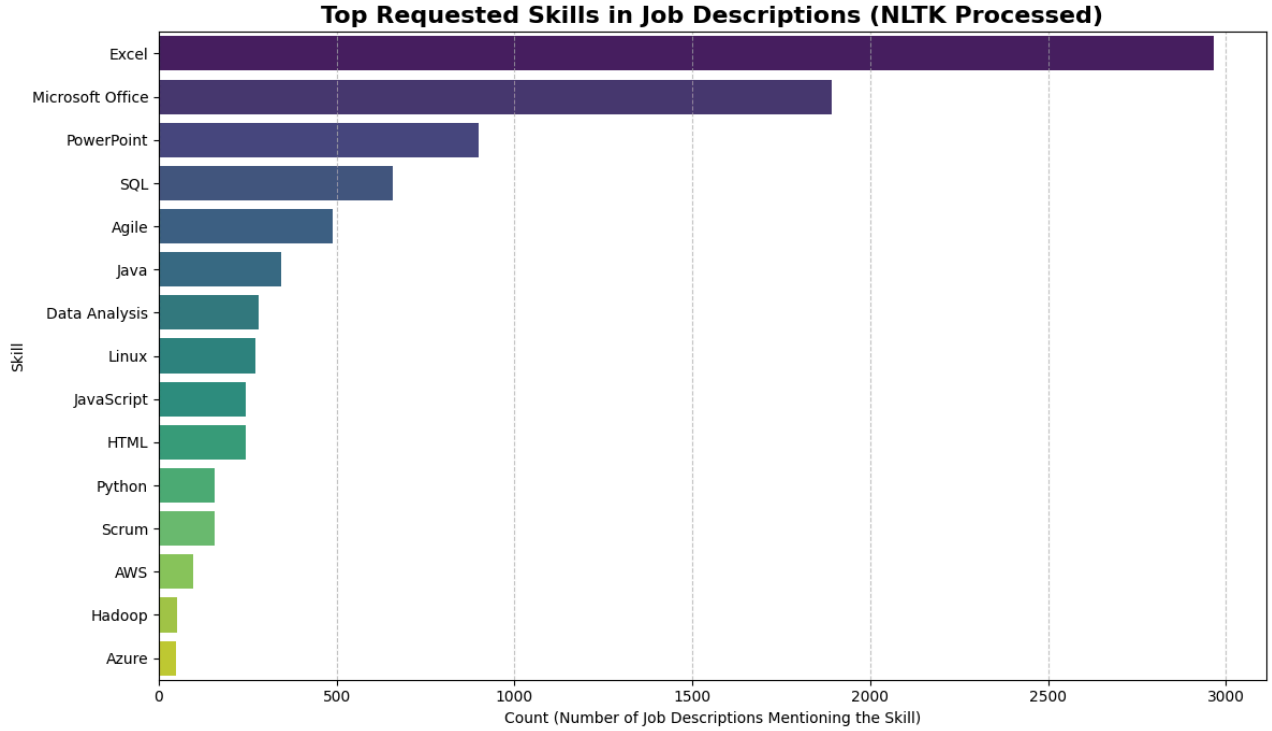
Findings:

- Words that stand out relating to jobs are Sales, Manger, Technician, Engineer, assistant as well as some others.
- This could indicate indemand jobs at the time.

```
# Load the dataset
# Assuming 'monster_com-job_sample.csv' is in the same directory
df_original = pd.read_csv("monster_com-job_sample.csv")

# Define the list of target skills (expanded for better coverage)
target_skills = [
    "Python", "SQL", "Excel", "Tableau", "R", "Machine Learning", "AWS",
    "Java", "C++", "Hadoop", "Spark", "Microsoft Office", "PowerPoint",
    "Data Analysis", "Statistics", "Deep Learning", "Azure", "GCP",
    "JavaScript", "HTML", "CSS", "C#", "Scrum", "Agile", "Linux"
]

# Initialize NLTK tools
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
skill_counts = Counter()


# --- 2. NLP Processing and Skill Extraction Function ---

def process_and_find_skills(text, target_skills, stop_words, lemmatizer):
    """Tokenizes, cleans, lemmatizes, and finds target skills in the text."""

    # Lowercase the text
    text = text.lower()

    # Remove special characters and numbers (optional, but helps cleaning)
    text = re.sub(r'[^a-z\s]', '', text)
```

```
    # Tokenize the text
    tokens = word_tokenize(text)

    # Remove stop words and lemmatize
    processed_tokens = [
        lemmatizer.lemmatize(w) for w in tokens
        if w not in stop_words and len(w) > 1
    ]

    # Check for skills
    found_skills = set()

    # 1. Check for multi-word skills (like 'Machine Learning')
    skill_map = {skill.lower(): skill for skill in target_skills}

    for skill_lower, skill_original in skill_map.items():
        if len(skill_lower.split()) > 1:
            if skill_lower in ' '.join(processed_tokens):
                found_skills.add(skill_original)
        else: # 2. Check for single-word skills
            if skill_lower in processed_tokens:
                found_skills.add(skill_original)

    return found_skills

# --- 3. Iterate and Count Skills ---

# Use .dropna() for safety
job_descriptions = df_original['job_description'].dropna().tolist()

print(f"Processing {len(job_descriptions)} job descriptions...")

for desc in job_descriptions:
    # Get the unique skills found in this description
    found_skills = process_and_find_skills(desc, target_skills, stop_words, lemmatizer)

    # Increment the counter for each found skill
    for skill in found_skills:
        skill_counts[skill] += 1

print("Processing complete.")
print(f"Total skills found: {sum(skill_counts.values())}")


# --- 4. Prepare Data for Visualization ---

# Convert to Series and get the top 15 skills
skill_series = pd.Series(skill_counts).sort_values(ascending=False).head(15)

# --- 5. Visualization (Graph) ---

plt.figure(figsize=(12, 7))
# Use barplot for clarity
sns.barplot(x=skill_series.values, y=skill_series.index, palette="viridis", hue=skill_series.index, le

plt.title("Top Requested Skills in Job Descriptions (NLTK Processed)",
          fontdict={'fontsize': 16, 'fontweight': 'bold'})
plt.xlabel("Count (Number of Job Descriptions Mentioning the Skill)")
plt.ylabel("Skill")
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
Processing 22000 job descriptions...
Processing complete.
Total skills found: 8929
```



Findings:

- Excel and Microsoft office are very popular skills and found the most based on the skills list made.
- Coding languages SQL and Java are the most in demand based on this dataset.
- Based on the results, I cannot say if a person should follow these guidelines. I do believe that some skills listed are accurate and should be learned if entering business field or any corporate environment such as Excel and Powerpoint. However, I think with tech skills, python is not where it should be as well as AWS to start.

EDA Summary

Based on the various visualizations and tables above there are some clear observations to be made about the dataset that can help when moving forward to the Modeling part of the project. First, based on the missing data, Salary is not a good field to proceed with as majority of the data in that field is missing. This will result in our modeling section being more of a classification problem to categorize job_titles/job_descriptions into job_categories. Then, locations are mostly in the south with some in the midwest and other regions, while this may not play a massive role in determining various other outcomes such as job_category or job_title it is still interesting to see and be able to draw conclusions from. Next, not all of the data is categorized properly resulting in most of the fields being other. The solution will be extracting words from job_title and catergorizing them into 10 different fields.

🛠️ Feature Engineering

To ensure model generalization and avoid "data leakage," the following steps were taken:

- **Semantic Truncation:** Job descriptions were capped at 500 characters to focus on the core "role mission" and reduce noise from footer text.
- **Handling Class Imbalance:** Identified that the 'Other' category acted as a "catch-all" bucket, necessitating advanced NLP techniques to resolve.
- **GPU-Accelerated Embeddings:** Leveraged `all-MiniLM-L6-v2` to map text into a 384-dimensional vector space, capturing context that traditional keyword counting misses.

```
from transformers import pipeline

# 1. Setup the 'Expert' model
import torch
device = 0 if torch.cuda.is_available() else -1

classifier = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli",
    device=device # This ensures it uses the T4 GPU you enabled
)

# 2. Pick 5 rows where your XGBoost predicted 'Other'
# Let's see if the LLM can find their TRUE home
other_samples = df[df['job_category'] == 'Other']['job_description'].iloc[:5].tolist()
candidate_labels = ["Technology", "Healthcare", "Finance", "Sales", "Legal"]

print("LLM is re-evaluating the 'Other' category...")
for text in other_samples:
    result = classifier(text[:500], candidate_labels) # Use first 500 chars
    print(f"Description Snippet: {text[:50]}...")
    print(f"LLM Suggestion: {result['labels'][0]} ({result['scores'][0]:.2f})")
    print("-" * 30)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json:         1.15k/? [00:00<00:00, 96.3kB/s]
model.safetensors: 100%                                                  1.63G/1.63G [00:10<00:00, 321MB/s]
tokenizer_config.json: 100%                                               26.0/26.0 [00:00<00:00, 3.32kB/s]
vocab.json:         899k/? [00:00<00:00, 29.6MB/s]
merges.txt:         456k/? [00:00<00:00, 23.7MB/s]
tokenizer.json:         1.36M/? [00:00<00:00, 47.7MB/s]
Device set to use cuda:0
LLM is re-evaluating the 'Other' category...
Description Snippet: Report this job About the Job DePuy Synthes Compan...
LLM Suggestion: Healthcare (0.59)
------------------------------
Description Snippet: Position ID#  76162 # Positions  1 State  CT City ...
LLM Suggestion: Legal (0.58)
------------------------------
Description Snippet: RESPONSIBILITIES:Kforce has a client seeking a Mai...
LLM Suggestion: Legal (0.36)
------------------------------
Description Snippet: Part-Time, 4:30 pm - 9:30 pm, Mon - Fri Brookdale ...
LLM Suggestion: Legal (0.51)
------------------------------
Description Snippet: Aflac Insurance Sales Agent While a career in sale...
LLM Suggestion: Sales (0.96)
------------------------------
```

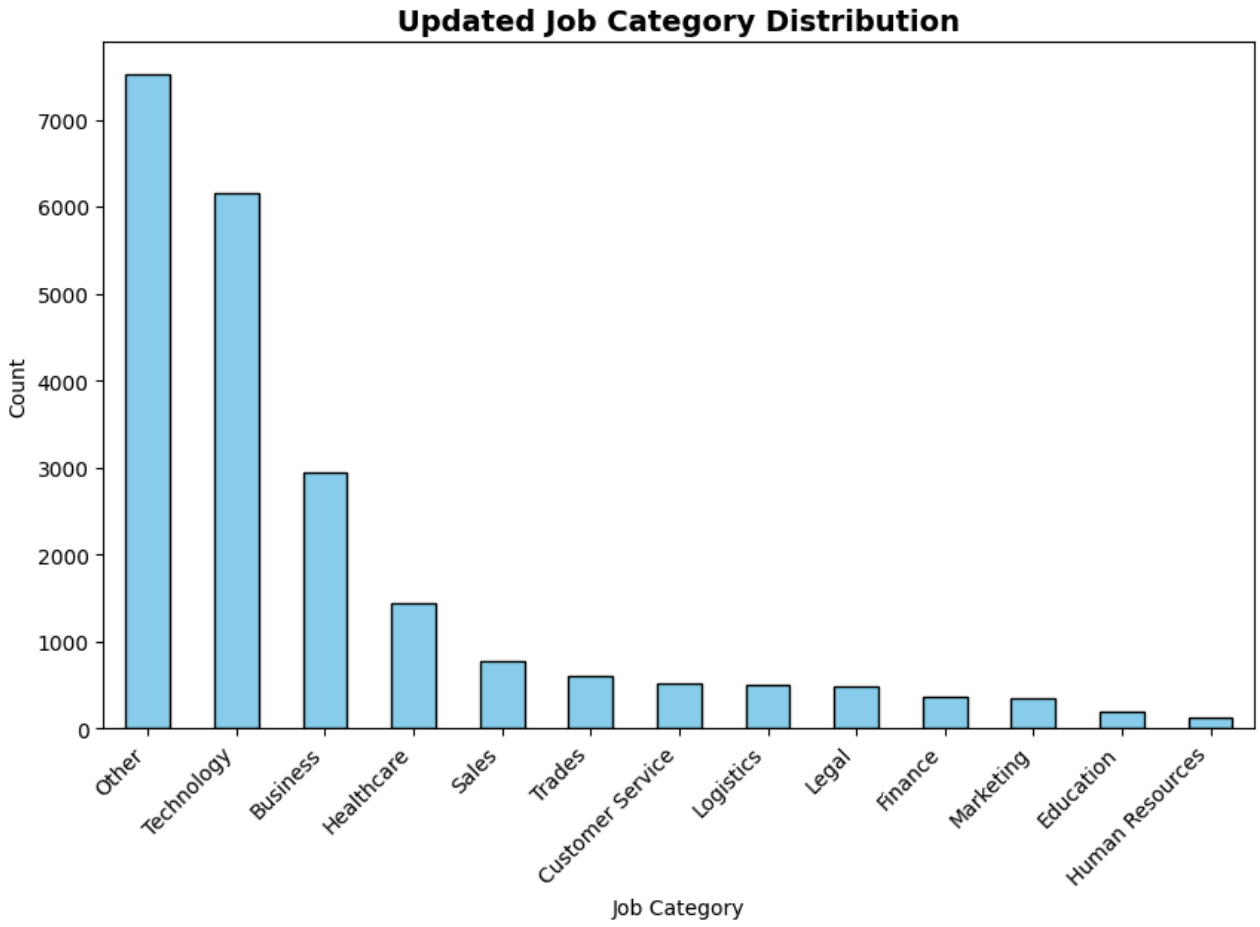
```
from transformers import pipeline
from tqdm.notebook import tqdm # For a nice progress bar

# 2. Filter for only the 'Other' rows to save time
other_df = df[df['job_category'] == 'Other'].copy()
descriptions = other_df['job_description'].str[:500].tolist() # Truncate to save memory
candidate_labels = ["Technology", "Healthcare", "Finance", "Sales", "Legal", "Education"]

# 3. Process in batches (this is much faster)
batch_size = 16
new_labels = []

print(f"Cleaning {len(descriptions)} rows in batches of {batch_size}...")

for i in tqdm(range(0, len(descriptions), batch_size), desc="LLM Re-labeling"):
    batch = descriptions[i : i + batch_size]
    results = classifier(batch, candidate_labels)

    # Only accept the new label if confidence is high (>0.7)
    for res in results:
        if res['scores'][0] > 0.7:
            new_labels.append(res['labels'][0])
        else:
            new_labels.append("Other") # Keep as 'Other' if the LLM is unsure

# 4. Map back to the dataframe
other_df['refined_industry'] = new_labels
```

```
Cleaning 9387 rows in batches of 16...
LLM Re-labeling: 100%                                                    587/587 [31:36<00:00,  2.92s/it]
You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a da
```

```
# Update the original df with the refined industries
df.loc[other_df.index, 'job_category'] = other_df['refined_industry']
```

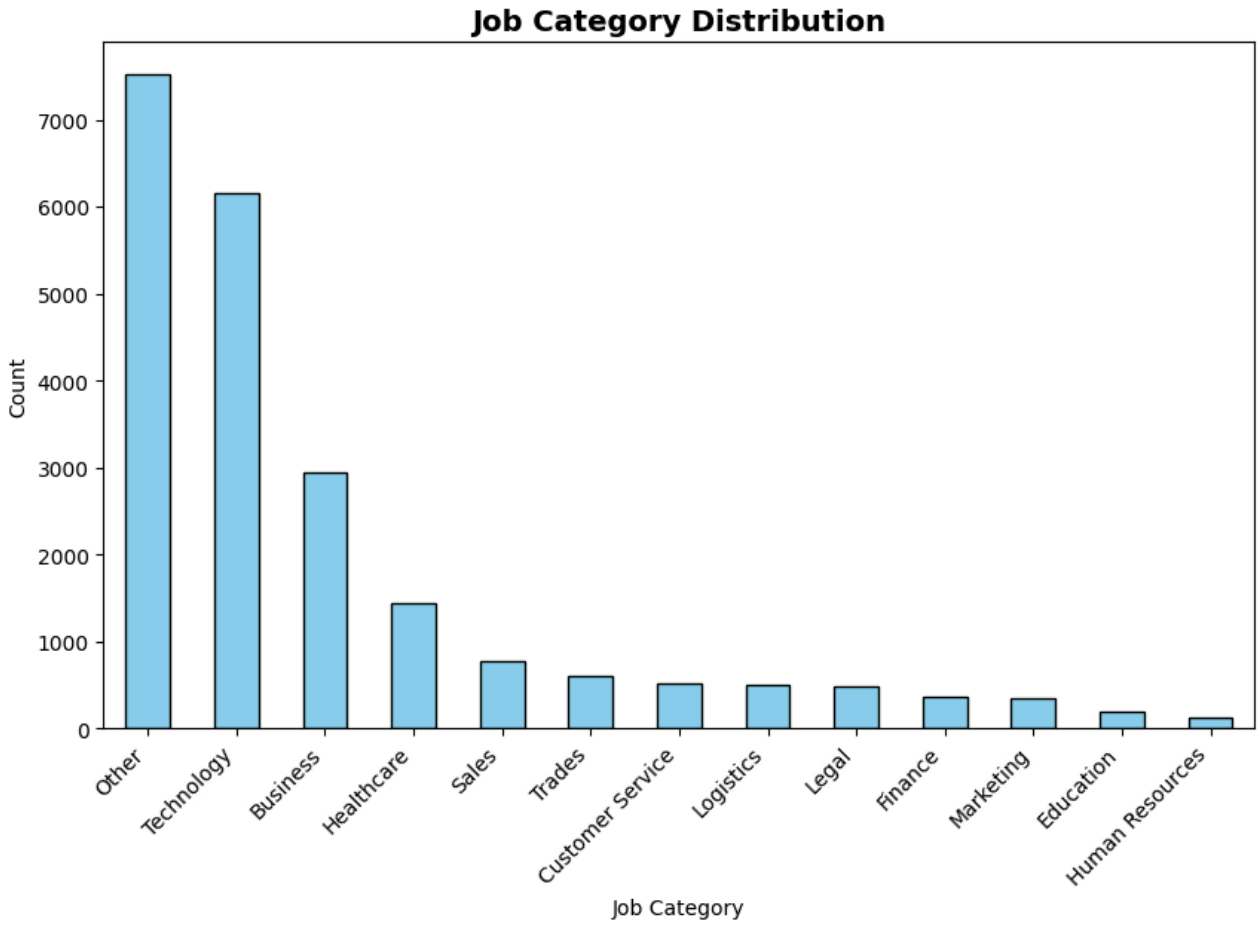
```
# Recount categories after updating
category_counts_updated = df['job_category'].value_counts()

# Print raw counts
print("Updated Job Category Counts:\n", category_counts_updated)

# Percentage distribution
print("\nUpdated Percentage Distribution:\n", (category_counts_updated / len(df) * 100).round(2))

# Plot distribution
plt.figure(figsize=(10,6))
category_counts_updated.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title("Updated Job Category Distribution",fontdict={'fontsize': 14, 'fontweight': 'bold'} )
plt.xlabel("Job Category")
plt.ylabel("Count")
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.show()
```

```
Updated Job Category Counts:
 job_category
Other               7514
Technology          6160
Business            2937
Healthcare          1434
Sales                780
Trades               599
Customer Service     527
Logistics            506
Legal                491
Finance              367
Marketing            348
Education            203
Human Resources      134
Name: count, dtype: int64

Updated Percentage Distribution:
 job_category
Other               34.15
Technology          28.00
Business            13.35
Healthcare           6.52
Sales                3.55
Trades               2.72
Customer Service     2.40
Logistics            2.30
Legal                2.23
Finance              1.67
Marketing            1.58
Education            0.92
Human Resources      0.61
Name: count, dtype: float64
```



```
# Count categories
category_counts = df['job_category'].value_counts()

# Print raw counts
print("Job Category Counts:\n", category_counts)

# Percentage distribution
print("\nPercentage Distribution:\n", (category_counts / len(df) * 100).round(2))

# Plot distribution
plt.figure(figsize=(10,6))
category_counts.plot(kind='bar', color='skyblue', edgecolor='black')
plt.title("Job Category Distribution",fontdict={'fontsize': 14, 'fontweight': 'bold'} )
plt.xlabel("Job Category")
plt.ylabel("Count")
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.show()
```

```
Job Category Counts:
 job_category
Other                7514
Technology           6160
Business             2937
Healthcare           1434
Sales                 780
Trades                599
Customer Service      527
Logistics             506
Legal                 491
Finance               367
Marketing             348
Education             203
Human Resources       134
Name: count, dtype: int64

Percentage Distribution:
 job_category
Other                34.15
Technology           28.00
Business             13.35
Healthcare            6.52
Sales                 3.55
Trades                2.72
Customer Service      2.40
Logistics             2.30
Legal                 2.23
Finance               1.67
Marketing             1.58
Education             0.92
Human Resources       0.61
Name: count, dtype: float64
```



Model Creation

```
#imports for models
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score, precisi
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
import pickle
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
import tensorflow as tf
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense, Embedding, GlobalMaxPooling1D, Dropout
from keras.preprocessing.sequence import pad_sequences
# XGBoost (make sure xgboost is installed)
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder

import joblib
import warnings
warnings.filterwarnings("ignore")

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

Necessary Imports to create various models such as RandomForest, XGBoost, and LogisticRegression as well as encoding the data

For this modeling section, the target feature will be job_category and the features used to predict this will be features that are not mostly empty as mentioned above when discussing the null/missing values.

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
descriptions = df['job_description'].fillna("None").tolist()
print("Generating embeddings... this is the 'Deep Learning' way to handle text!")
embeddings = model.encode(descriptions, show_progress_bar=True)
embedding_df = pd.DataFrame(embeddings)
embedding_df.columns = [f'emb_{i}' for i in range(embedding_df.shape[1])]
```

```
modules.json: 100%  349/349 [00:00<00:00, 41.2kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 13.4kB/s]
README.md: 10.5k/? [00:00<00:00, 887kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 6.51kB/s]
config.json: 100%  612/612 [00:00<00:00, 67.3kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 82.8MB/s]
tokenizer_config.json: 100%  350/350 [00:00<00:00, 38.7kB/s]
vocab.txt: 232k/? [00:00<00:00, 11.1MB/s]
tokenizer.json: 466k/? [00:00<00:00, 32.7MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 9.97kB/s]
config.json: 100%  190/190 [00:00<00:00, 18.1kB/s]
Generating embeddings... this is the 'Deep Learning' way to handle text!
Batches: 100%  688/688 [01:19<00:00, 38.07it/s]
```

```
RANDOM_STATE = 42

# 1) Split (use raw text column name 'job_description' or adapt)
#X_text = df['job_description'].fillna('')   # adjust column name
y = df['job_category']                   # adjust label column name

from sklearn.model_selection import train_test_split
X_train_text, X_test_text, y_train, y_test = train_test_split(
    embedding_df, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE
)
```

```
# # 2) TF-IDF
# from sklearn.feature_extraction.text import TfidfVectorizer
# tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2), min_df=5, max_df=0.85, stop_words='eng
# tfidf.fit(X_train_text)
# X_train_tfidf = tfidf.transform(X_train_text)
# X_test_tfidf  = tfidf.transform(X_test_text)

# # Encode target labels to numerical values for XGBoost
# label_encoder = LabelEncoder()
# y_train_encoded = label_encoder.fit_transform(y_train)
# y_test_encoded = label_encoder.transform(y_test)
# print("Encoded y_train_encoded unique values:", np.unique(y_train_encoded))
# print("Original y_train unique values:", np.unique(y_train))
```

Using Label Encoding to format string values into numeric format for machine learning models. Applying vectorizer towards the data to transform the data.

```
#print("Train size:", X_train_tfidf.shape, "Test size:", X_test_tfidf.shape)
```

Logistic Regression

```
from sklearn.pipeline import Pipeline

#Creating logistic regression model
```

```
# Redefine log_reg_pipeline to only contain the classifier,
# as preprocessing is done separately before SMOTE.
log_reg_clf = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=RANDOM_STATE,
    n_jobs=-1 if "n_jobs" in LogisticRegression().get_params() else None
)
```

```
#Fitting training data (resampled) and predicting test data (processed)
log_reg_clf.fit(X_train_text, y_train)
y_pred = log_reg_clf.predict(X_test_text)
#y_pred = label_encoder.inverse_transform(y_pred)
```

```
#Print metrics on model performance
print("Classification Report:")
print(classification_report(y_test, y_pred, digits=3))
```

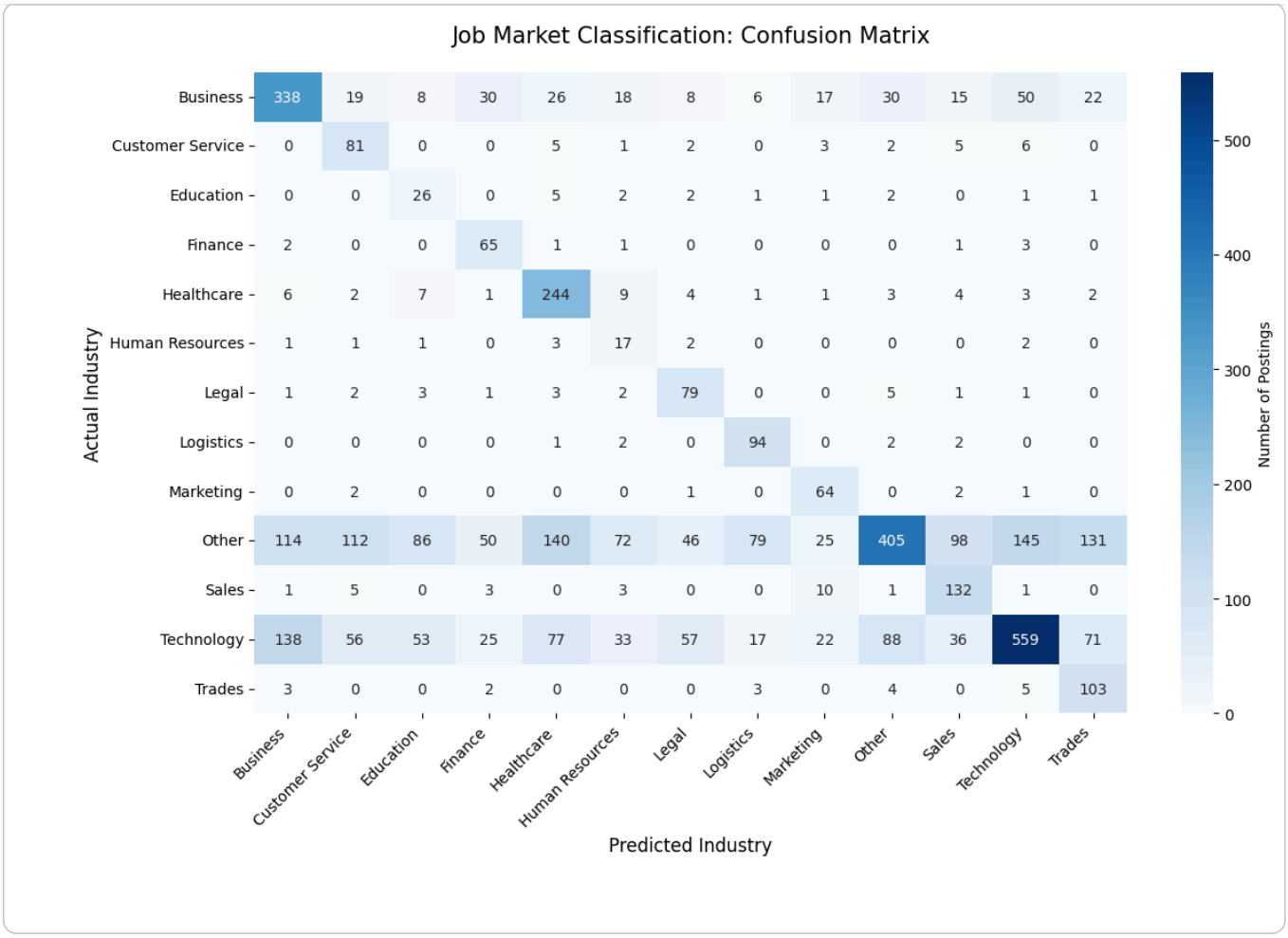
```
Classification Report:
                   precision    recall  f1-score   support

         Business      0.560     0.576     0.568       587
Customer Service       0.289     0.771     0.421       105
        Education      0.141     0.634     0.231        41
          Finance      0.367     0.890     0.520        73
       Healthcare      0.483     0.850     0.616       287
 Human Resources       0.106     0.630     0.182        27
            Legal      0.393     0.806     0.528        98
        Logistics      0.468     0.931     0.623       101
        Marketing      0.448     0.914     0.601        70
            Other      0.747     0.269     0.396      1503
            Sales      0.446     0.846     0.584       156
       Technology      0.719     0.454     0.556      1232
           Trades      0.312     0.858     0.458       120

         accuracy                          0.502      4400
        macro avg      0.422     0.725     0.483      4400
     weighted avg      0.629     0.502     0.498      4400
```

```
import shap
explainer = shap.LinearExplainer(log_reg_clf, X_train_text)
shap_values = explainer.shap_values(X_test_text)
```

```
# Sample 100 random jobs to show a variety of industries
from sklearn.manifold import TSNE
sample_df = df.sample(100, random_state=42)
sample_titles = sample_df['job_title']
sample_embeddings = model.encode(sample_df['job_description'].tolist()) # Re-encoding a small batch

tsne = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(sample_embeddings)

plt.figure(figsize=(14, 10))
# Using a color map based on the category makes the clusters pop
for i, title in enumerate(sample_titles):
    plt.scatter(tsne[i, 0], tsne[i, 1], alpha=0.7)
    plt.text(tsne[i, 0] + 0.2, tsne[i, 1] + 0.2, title, fontsize=8, alpha=0.8)

plt.title("Semantic Mapping: How the Model 'Sees' Job Relationships", fontsize=15, pad=20)
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
```

```
plt.grid(True, linestyle='--', alpha=0.3)
plt.show()
```


```
cmlr = confusion_matrix(y_test, y_pred)

# 2. Get the unique category names (labels)
# It's important they are in the same order as the matrix
labels = sorted(y_test.unique())

# 3. Set up the figure size - Make it wide enough to read!
plt.figure(figsize=(12, 8))

# 4. Create the heatmap
sns.heatmap(
    cmlr,
    annot=True,               # Put the numbers in the boxes
    fmt='d',                  # Use integers instead of scientific notation
    cmap='Blues',             # High contrast color scheme
    xticklabels=labels,
    yticklabels=labels,
    cbar_kws={'label': 'Number of Postings'}
)

# 5. Fix the labels (The most important part for readability!)
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(rotation=0, fontsize=10)

plt.title('Job Market Classification: Confusion Matrix', fontsize=15, pad=20)
plt.xlabel('Predicted Industry', fontsize=12)
plt.ylabel('Actual Industry', fontsize=12)

plt.tight_layout() # Ensures labels aren't cut off
plt.show()
```



Findings:

- Overall performance is good but not spectacular as there are some job_categories being misclassified
- This can be due to not enough data from in those other fields. Could also be because the fields also have similar skills or job_descriptions.
- Categories other and technology have the most that are misclassified with every other category but also have most of the data.
- Other may be misclassifying as those positions might be related to the other fields.

Random Forest Model

```
#Create RandomForest model
# Redefine rf_clf to only contain the classifier, as preprocessing is done separately.
rf_clf = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    class_weight="balanced",
    random_state=RANDOM_STATE,
    n_jobs=-1
)
```

Create and initialize the RandomForest Classifier

```
#Fit train data (resampled), and predict test data (processed)
rf_clf.fit(X_train_text, y_train)
```

```
y_pred_rf = rf_clf.predict(X_test_text)

print("Random Forest - Classification Report:")
print(classification_report(y_test, y_pred_rf, digits=3))
```

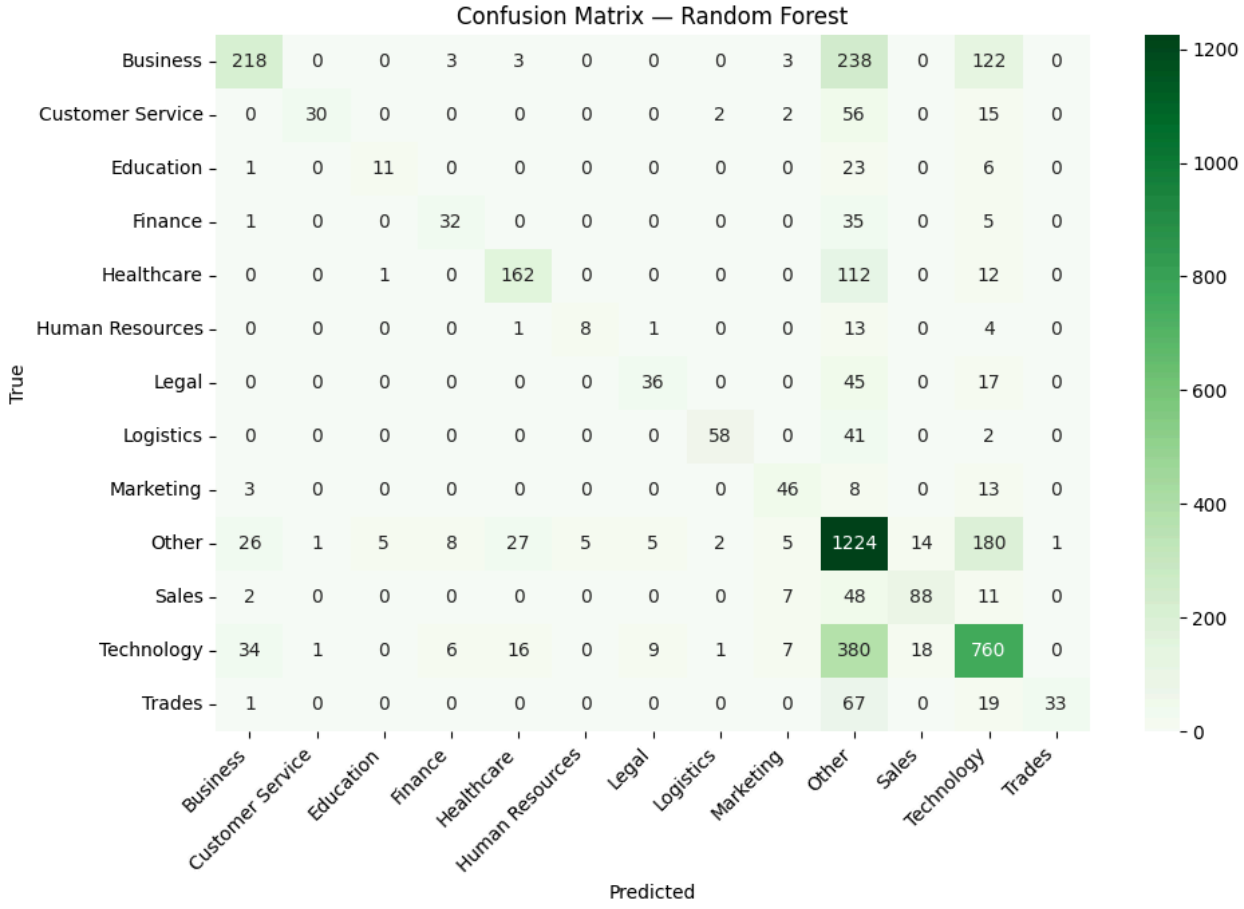
```
Random Forest - Classification Report:
                  precision    recall  f1-score   support

        Business      0.762     0.371     0.499       587
Customer Service      0.938     0.286     0.438       105
       Education      0.647     0.268     0.379        41
         Finance      0.653     0.438     0.525        73
      Healthcare      0.775     0.564     0.653       287
 Human Resources      0.615     0.296     0.400        27
           Legal      0.706     0.367     0.483        98
       Logistics      0.921     0.574     0.707       101
       Marketing      0.657     0.657     0.657        70
           Other      0.534     0.814     0.645      1503
           Sales      0.733     0.564     0.638       156
      Technology      0.652     0.617     0.634      1232
          Trades      0.971     0.275     0.429       120

        accuracy                          0.615      4400
       macro avg      0.736     0.469     0.545      4400
    weighted avg      0.660     0.615     0.604      4400
```

```
# Cross-validation setup
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Cross-validate Random Forest
# Use the processed X_train and y_train for cross-validation within the pipeline
# Note: This is now just cross-validating the RF classifier on already processed data, not the full pre
# For a full pipeline CV, `imblearn.pipeline.Pipeline` would be needed.
rf_cv_scores = cross_val_score(rf_clf, X_train_text, y_train, cv=cv, scoring="accuracy")
print("Random Forest CV Mean Accuracy:", rf_cv_scores.mean())
print("Random Forest CV Scores:", rf_cv_scores)
```

```
Random Forest CV Mean Accuracy: 0.6063068181818182
Random Forest CV Scores: [0.60454545 0.60795455 0.61789773 0.603125   0.59801136]
```

```
#Print metric scores and confusion matrix
print("RF — Accuracy:", accuracy_score(y_test, y_pred_rf))
print("RF — Macro F1:", f1_score(y_test, y_pred_rf, average="macro"))
print("\nClassification report (RF):\n", classification_report(y_test, y_pred_rf, digits=3))

cm_rf = confusion_matrix(y_test, y_pred_rf, labels=rf_clf.classes_)
plt.figure(figsize=(10,7))
sns.heatmap(cm_rf, annot=True, fmt="d", cmap="Greens",
            xticklabels=rf_clf.classes_, yticklabels=rf_clf.classes_)
plt.title("Confusion Matrix — Random Forest")
plt.xlabel("Predicted"); plt.ylabel("True")
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout(); plt.show()
```

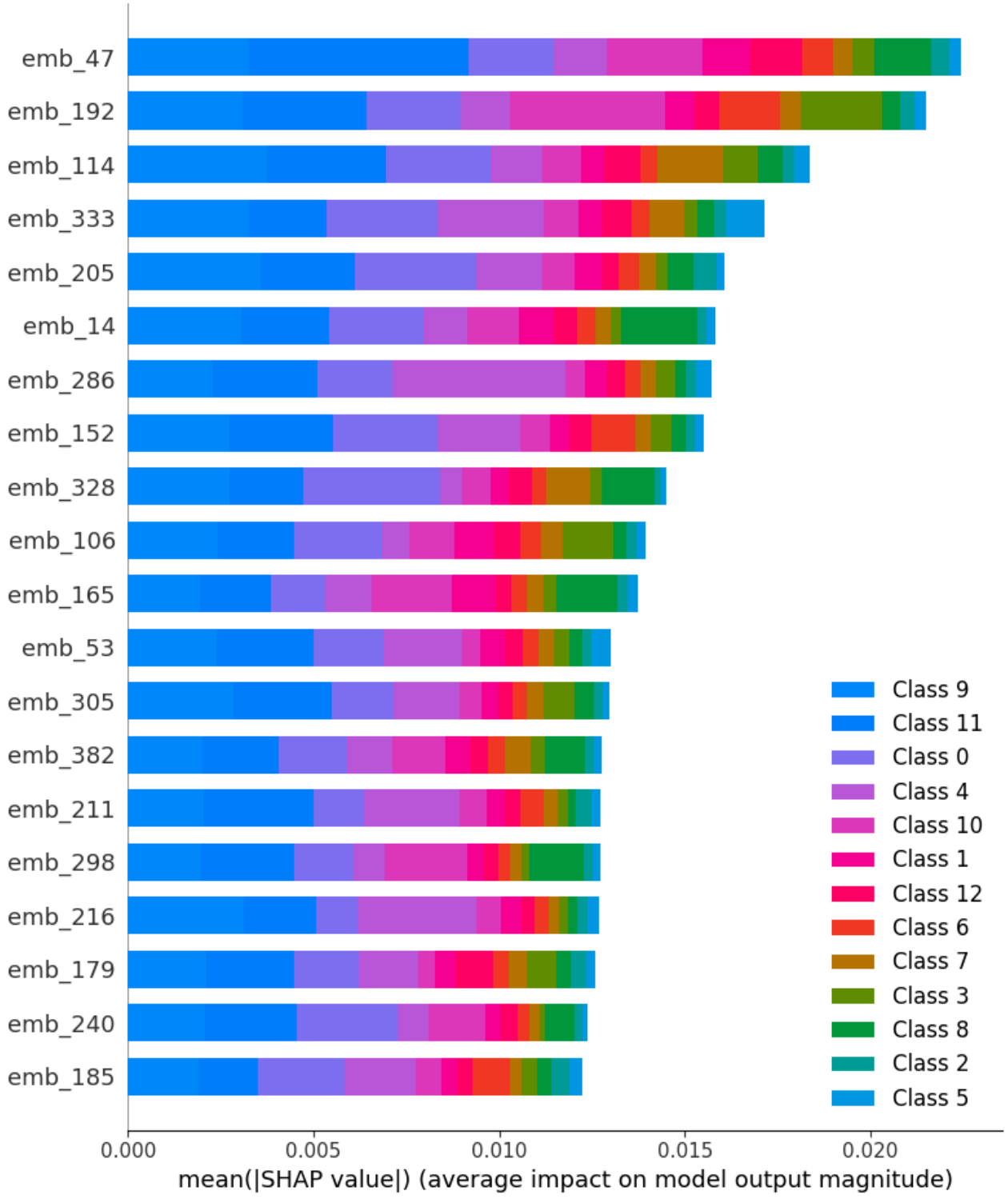
```
RF — Accuracy: 0.615
RF — Macro F1: 0.5452080575384114

Classification report (RF):
                    precision    recall  f1-score   support

          Business       0.762     0.371     0.499       587
Customer Service         0.938     0.286     0.438       105
         Education       0.647     0.268     0.379        41
           Finance       0.653     0.438     0.525        73
        Healthcare       0.775     0.564     0.653       287
 Human Resources         0.615     0.296     0.400        27
             Legal       0.706     0.367     0.483        98
         Logistics       0.921     0.574     0.707       101
         Marketing       0.657     0.657     0.657        70
             Other       0.534     0.814     0.645      1503
             Sales       0.733     0.564     0.638       156
        Technology       0.652     0.617     0.634      1232
            Trades       0.971     0.275     0.429       120

          accuracy                           0.615      4400
         macro avg       0.736     0.469     0.545      4400
      weighted avg       0.660     0.615     0.604      4400
```




```
shap_sample_size = 1000  # Define a smaller sample size for SHAP explanation
X_test_shap_sample = X_test_text.sample(shap_sample_size, random_state=RANDOM_STATE)

explainer_rf = shap.TreeExplainer(rf_clf, X_train_text)
# Fix: Disable the additivity check as suggested by the error message.
# This prevents the ExplainerError when minor numerical discrepancies occur.
shap_values_rf = explainer_rf.shap_values(X_test_shap_sample, check_additivity=False)
```

```
print("Generating SHAP Summary Plot for Random Forest...")
# This bar plot shows the mean absolute SHAP value for each feature across all classes
shap.summary_plot(shap_values_rf, X_test_shap_sample, plot_type="bar")
```

```
100%|==================| 12998/13000 [28:43<00:00]       Generating SHAP Summary Plot for Random Fores
```




```
# Detailed class-by-class metrics
prec, rec, f1, _ = precision_recall_fscore_support(y_test, y_pred_rf, average=None)

print("\n🔹 Detailed Metrics per Class (RF)")
for i, label in enumerate(rf_clf.classes_):
    print(f"{label}: Precision={prec[i]:.2f}, Recall={rec[i]:.2f}, F1={f1[i]:.2f}")
```

```
🔹 Detailed Metrics per Class (RF)
Business: Precision=0.76, Recall=0.37, F1=0.50
Customer Service: Precision=0.94, Recall=0.29, F1=0.44
Education: Precision=0.65, Recall=0.27, F1=0.38
Finance: Precision=0.65, Recall=0.44, F1=0.52
Healthcare: Precision=0.78, Recall=0.56, F1=0.65
Human Resources: Precision=0.62, Recall=0.30, F1=0.40
Legal: Precision=0.71, Recall=0.37, F1=0.48
Logistics: Precision=0.92, Recall=0.57, F1=0.71
Marketing: Precision=0.66, Recall=0.66, F1=0.66
Other: Precision=0.53, Recall=0.81, F1=0.65
Sales: Precision=0.73, Recall=0.56, F1=0.64
Technology: Precision=0.65, Recall=0.62, F1=0.63
Trades: Precision=0.97, Recall=0.28, F1=0.43
```

Findings:

- Better performance than the Logisitic Regression Model
- Classifies better in all areas as compared to the Logistic Regression Model
- Still misclassifications when it comes to other but as mentioned before it may be due to those positions being in the other fields.

Improving RF model

XGBoost

```
import xgboost as xgb
```

```
le = LabelEncoder()

# 2. Fit and transform your target labels
y_train_encoded = le.fit_transform(y_train)
y_test_encoded = le.transform(y_test)
```

```
xgb_model = xgb.XGBClassifier(
    n_estimators=100,
    max_depth=3,
    learning_rate=0.1
)
import warnings
# Silence non-critical warnings for the final export
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    xgb_model.fit(X_train_text, y_train_encoded)
```

```
pred_probs = xgb_model.predict(X_test_text)
y_pred_xgb = le.inverse_transform(pred_probs)
```

```
print("XGB — Accuracy:", accuracy_score(y_test, y_pred_xgb))
print("XGB — Macro F1:", f1_score(y_test, y_pred_xgb, average="macro"))
print("\nClassification report (XGB):\n", classification_report(y_test, y_pred_xgb, digits=3))
```

```
XGB — Accuracy: 0.6145454545454545
XGB — Macro F1: 0.5538582728179937

Classification report (XGB):
```

```
                    precision    recall  f1-score   support

          Business       0.620     0.470     0.535       587
Customer Service       0.674     0.295     0.411       105
         Education       0.800     0.195     0.314        41
           Finance       0.600     0.616     0.608        73
        Healthcare       0.618     0.666     0.641       287
 Human Resources         0.667     0.222     0.333        27
             Legal       0.671     0.520     0.586        98
         Logistics       0.867     0.644     0.739       101
         Marketing       0.591     0.557     0.574        70
             Other       0.572     0.753     0.650      1503
             Sales       0.729     0.603     0.660       156
        Technology       0.641     0.581     0.610      1232
            Trades       0.769     0.417     0.541       120

          accuracy                           0.615      4400
         macro avg       0.678     0.503     0.554      4400
      weighted avg       0.627     0.615     0.608      4400
```

```
# Evaluate on test
# Decode predictions back to original string labels for comparison with y_test
cm_xgb = confusion_matrix(y_test, y_pred_xgb, labels=le.classes_)
plt.figure(figsize=(12, 8))
sns.heatmap(
    cm_xgb,
    annot=True,
    fmt="d",
    cmap="Oranges",
    xticklabels=le.classes_,
    yticklabels=le.classes_,
    cbar_kws={'label': 'Count'}
)

# Crucial for readability: ha="right" prevents text overlap
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.title("Confusion Matrix: Neural Embedding + XGBoost", pad=20, fontsize=14)
plt.tight_layout()
plt.show()
```